

Universidad Nacional Autónoma de México

Facultad de Ciencias

Lenguajes de Programación

Proyecto 1:MiniLisp

Integrantes:

Elizalde Maza Jesús Eduardo
Navarro Fierro Michelle Alanis
Peredo López Citlalli Abigail

Profesor: Manuel Soto Romero

Diego Méndez Medina
José Alejandro Pérez Márquez
Erick Daniel Arroyo Martínez
Mauro Emiliano Chávez Zamora

Fecha: 1 de diciembre de 2025

CDMX

Índice

1. Introducción	3
2. Objetivos	4
3. Sintaxis de un lenguaje de programación	5
3.1. Sintaxis concreta	5
3.1.1. Sintaxis Léxica	5
3.2. Gramática Formal: gramática libre de contexto (GLC)	9
3.2.1. Notación EBNF	13
4. Sintaxis Abstracta	16
4.1. Árboles de sintaxis	16
4.1.1. Árboles de sintaxis concreta	17
4.1.2. Árboles de sintaxis abstracta (ASA)	19
5. Azúcar sintáctica	24
5.1. Eliminación de azúcar sintáctica: el Proceso de Desugaring en LAMBDAE	26
5.1.1. SASA: Sintaxis Rica con Azúcar Sintáctica	26
5.1.2. ASA: Lenguaje Núcleo Desugared	29
5.2. Transformaciones de Desugaring en LAMBDAE	29
5.2.1. Desugaring de Operaciones N-arias	29
5.2.2. Desugaring de Funciones Lambda Multi-parámetro	29
5.2.3. Desugaring de Construcciones Let	30
5.2.4. Desugaring de Let*	30
5.2.5. Desugaring de Expresiones Condicionales Múltiples	30
5.2.6. Desugaring de Azúcar de Conveniencia	31
6. Desazucaramiento de Operadores Variádicos	32
6.1. El Papel Fundamental de fold	32
6.2. Operadores Aritméticos	33
7. Importancia de foldl1 y foldr1	34
7.1. Asociatividad Izquierda en Operadores Aritméticos	34
7.2. Exponenciación: Asociatividad Derecha	34
7.3. Comparaciones Encadenadas	34
8. Implementación	35
9. Currying: Lambdas Multivariádicas a Unarias	35
9.1. Principio de Currying	35
9.2. Aplicación Multivariádica	36
10. Desazucaramiento de Enlaces	36
10.1. Let Simple	36
10.2. let*	37
10.3. letrec y el Combinador Z	37

11.Listas y Desazucaramiento	38
11.1. Construcción	38
12.Desazucaramiento de cond	38
12.1. Beneficios Arquitectónicos del Desugaring en LAMBDAE	39
12.1.1. Simplificación del Evaluador	39
12.1.2. Reutilización de Código y Demostraciones	39
12.1.3. Extensibilidad del Lenguaje	40
12.1.4. Claridad Semántica	40
13.Semántica operacional	40
13.1. Reglas Básicas	42
14.Conclusiones	51
15.Referencias	53

1. Introducción

En la actualidad, los lenguajes de programación constituyen una de las herramientas más fundamentales en el campo de las Ciencias de la Computación, ya que no solo permiten escribir código, sino que representan el medio por el cual los seres humanos comunican instrucciones complejas a las computadoras. A través de ellos es posible expresar algoritmos e ideas de forma precisa y estructurada, impulsando el desarrollo de sistemas operativos, aplicaciones móviles, inteligencia artificial y una amplia gama de tecnologías que forman parte esencial de la vida cotidiana. En términos generales, un lenguaje de programación puede definirse como un conjunto de reglas y símbolos que posibilitan la escritura de instrucciones comprensibles para la máquina.

La historia de los lenguajes de programación refleja la búsqueda constante por lograr una mayor abstracción, eficiencia y correctitud en la comunicación entre humanos y computadoras. En sus inicios, la programación se realizaba mediante lenguajes de bajo nivel, como el código de máquina y el ensamblador, los cuales resultaban difíciles de leer y propensos a errores. La aparición de lenguajes de alto nivel como FORTRAN y COBOL marcó un punto de inflexión, al permitir a los programadores centrarse en la lógica del problema en lugar de los detalles de la arquitectura del hardware. [17, 18]

Posteriormente, la evolución de los lenguajes dio lugar a diversos paradigmas que transformaron la manera de desarrollar software. La programación estructurada, con lenguajes como Pascal y C, promovió la organización del código en bloques lógicos, reduciendo la complejidad y mejorando la legibilidad. Más adelante, la programación orientada a objetos, representada por C++, Java y Smalltalk, introdujo conceptos como la encapsulación, la herencia y el polimorfismo, permitiendo modelar problemas del mundo real de forma más intuitiva y fomentando la creación de sistemas modulares y reutilizables. En paralelo, la programación funcional, impulsada por lenguajes como Lisp, Haskell y Scala, cobró relevancia por su capacidad para manejar la concurrencia y el procesamiento de datos, gracias a principios como la inmutabilidad y las funciones puras, que facilitan el razonamiento y las pruebas del código. [19]

Esta evolución constante demuestra que los lenguajes de programación no solo han acompañado el progreso tecnológico, sino que han sido motores fundamentales en la transformación de la informática moderna, respondiendo a la necesidad de abordar problemas cada vez más complejos y especializados.

En este contexto, el presente proyecto se enmarca en el estudio formal de los lenguajes de programación mediante la extensión de un lenguaje de estilo Lisp denominado MiniLisp, implementado en Haskell con apoyo de la herramienta Happy. Este trabajo busca recorrer las etapas esenciales de la formalización de un lenguaje, desde la definición de su sintaxis léxica y libre de contexto hasta la especificación de su semántica operacional, con el propósito de integrar la teoría y la práctica en el diseño de intérpretes. De esta manera, el proyecto MiniLisp constituye una oportunidad para aplicar los fundamentos teóricos de los lenguajes de programación en un ambiente funcional real, fortaleciendo la comprensión de los principios que rigen su construcción, análisis e implementación.

2. Objetivos

El presente proyecto tiene como propósito definir formalmente la sintaxis léxica y libre de contexto del lenguaje MiniLisp, por lo que se propone el lenguaje LAMBDAE que es una extensión del mismo. Asimismo se formula su representación en sintaxis abstracta e incorporando mecanismos de azúcar sintáctica y su correspondiente eliminación hacia un núcleo mínimo. Por otra parte, se busca diseñar y especificar la semántica operacional estructural del lenguaje mediante reglas de inferencia y derivaciones que describan con precisión su comportamiento durante la evaluación, al tiempo que se modelan ambientes de evaluación y bindings que garanticen la correcta gestión de alcances, valores y consistencia semántica.

Con el fin de ampliar la expresividad del lenguaje, se propone incorporar nuevos operadores aritméticos, predicados, estructuras de datos, listas, condicionales y funciones variádicas, sustentando formalmente cada decisión de diseño. De igual manera, se plantea la implementación de un intérprete funcional completo de MiniLisp en Haskell, estructurado a partir de un pipeline modular que abarque las fases de análisis léxico, sintáctico, desazucarización y evaluación, asegurando una correspondencia directa entre la teoría formal y la ejecución práctica.

Finalmente, se busca demostrar la validez del modelo propuesto mediante casos de prueba y ejemplos representativos que evidencien la relación entre la formalización teórica y el comportamiento del sistema implementado, así como integrar y documentar los resultados en un informe académico que articule los fundamentos teóricos, las decisiones de diseño y los resultados experimentales, promoviendo una comprensión crítica del vínculo entre teoría y práctica en el estudio de los lenguajes de programación.

3. Sintaxis de un lenguaje de programación

En el diseño de los lenguajes de programación, establecer reglas claras que indiquen cómo deben escribirse y estructurarse los programas es un aspecto esencial. Estas reglas no sólo permiten que las máquinas comprendan las instrucciones, sino que también aseguran que los desarrolladores puedan comunicarse de manera precisa y coherente con ellas. Sin una estructura formal que determine qué expresiones son válidas y cómo deben organizarse, sería imposible garantizar que un programa sea interpretado de la misma manera por distintos compiladores o intérpretes. Además, esta estructura facilita la detección de errores, la lectura del código y la creación de herramientas de análisis y depuración. En este contexto surge la necesidad de definir con rigor la forma en que los programas deben escribirse, dando origen al concepto fundamental de la sintaxis en los lenguajes de programación.

Definición: Sintaxis de un lenguaje de programación

Definimos la sintaxis de un lenguaje de programación como el conjunto de reglas y estructuras que especifican cómo deben escribirse los símbolos y palabras reservadas de un programa para que sea aceptado por el traductor del lenguaje. [1, 4]

3.1. Sintaxis concreta

La sintaxis concreta de un lenguaje de programación define la apariencia y estructura exacta con la que deben escribirse los programas, abarcando todos los elementos visibles como palabras reservadas, operadores, signos de puntuación y la forma en que estos se organizan. Por lo que por eso definir a la sintaxis concreta de la siguiente forma:

Definición: Sintaxis concreta

La sintaxis concreta se puede definir como un par $(\mathbf{L}, \mathbf{G})$ donde:

- $\mathbf{L}$ es la definición léxica representada por el conjunto de expresiones regulares $\mathbf{R}$.
- $\mathbf{G}$ es la gramática libre de contexto que se ve representada por una cuatrupla de la siguiente forma: $(\mathbf{N}, \Sigma, \mathbf{P}, \mathbf{S})$. [5, 20]

Dicha sintaxis se especifica mediante una gramática formal que determina, qué combinaciones de símbolos son válidas dentro del lenguaje.

Esta descripción se divide normalmente en dos niveles:

- Sintaxis léxica.
- Sintaxis estructural o gramática libre de contexto.

A continuación abordaremos un poco más a fondo cada uno de estos conceptos.

3.1.1. Sintaxis Léxica

Los elementos léxicos de un lenguaje de programación conforman el conjunto de símbolos, palabras reservadas y reglas de formación que determinan la estructura básica

mediante la cual se construyen las expresiones y sentencias del lenguaje. Este conjunto de caracteres reconocidos por el compilador o intérprete recibe el nombre de tokens, los cuales representan las unidades léxicas fundamentales del lenguaje. Dichos tokens pueden corresponder a símbolos de puntuación, operadores, representaciones numéricas, cadenas de texto o identificadores de variables.

Desde un punto de vista formal, la descripción léxica de un lenguaje se fundamenta en ciertos elementos esenciales:

- **Alfabeto (Σ):** se refiere al conjunto limitado de símbolos que pueden utilizarse en el lenguaje, como letras, números y signos especiales.
- **Tokens:** son secuencias de caracteres derivadas del alfabeto que representan las unidades sintácticas más simples. Cada clase de token se especifica mediante una expresión regular que determina su estructura o patrón.

Definición: Expresiones regulares

Una expresión regular E , definida sobre un alfabeto Σ , se construye de manera recursiva siguiendo las reglas siguientes:

- ε , que representa la cadena vacía, es considerada una expresión regular.
- Para cualquier símbolo $a \in \Sigma$, dicho símbolo constituye por sí mismo una expresión regular que representa una cadena formada únicamente por a .
- Si E_1 y E_2 son expresiones regulares, entonces su concatenación $E_1 E_2$ también forma una expresión regular.
- De igual manera, si E_1 y E_2 son expresiones regulares, su unión $E_1 + E_2$ se considera una expresión regular.
- Si E es una expresión regular, su cerradura de Kleene E^* también lo es, y describe cualquier número de repeticiones de E , incluyendo la posibilidad de que no aparezca. [5, 20, 21]

En consecuencia, para conformar el conjunto de tokens de un lenguaje, el analizador léxico recibe una secuencia de cadenas de entrada y las transforma en un conjunto de tokens válidos conforme a las reglas léxicas previamente definidas para dicho lenguaje.

El conjunto de tokens aceptados por nuestro lenguaje puede entenderse como una lista de unidades léxicas generada a partir de la lectura secuencial del programa de entrada, carácter por carácter. Durante este proceso, el analizador léxico identifica secuencias de caracteres que coinciden con alguno de los patrones o tokens previamente definidos en el lexer, y cada vez que se reconoce una coincidencia, se genera un nuevo token que se añade a la lista. De esta manera, al finalizar la lectura completa del programa, se obtiene un conjunto estructurado de tokens que representa de forma simbólica el contenido del código fuente.

A continuación, presentamos la especificación léxica del lenguaje LAMBDAE, definiendo formalmente su alfabeto y los principales tipos de tokens utilizados.

Corrección: Sintaxis léxica de LAMBDAE

Para comenzar a definir la sintaxis léxica comenzaremos definiendo el alfabeto de nuestro lenguaje, el cual está conformado por lo siguiente:

$$\Sigma = \{ \textit{True}, \textit{False}, +, -, *, /, \textit{sqrt}, \textit{expt}, \textit{not}, \textit{null?}, (,), (,)^1, [,], =, >, <, !, \textit{if}, \textit{fst}, \textit{snd}, \textit{let}, \textit{let}^*, \textit{letrec}, \textit{lamdba}, \textit{head}, \textit{tail}, \textit{pair}, \textit{cond}, \textit{add1}, \textit{sub1} \}$$

En nustra gramática también tenemos definidos a los enteros y a las variables. A las varibales las identificaremos como *id* y a los números como *int*, a ambos los podemos definir de la siguiente manera:

id ::= [A-Za-z][A-Za-z0-9]*

int ::= 0 | [1-9][0-9]*

bool ::= #t | #f

Palabras reservadas

<i>if</i>	::= <i>if</i>
<i>fst</i>	::= <i>fst</i>
<i>snd</i>	::= <i>snd</i>
<i>let</i>	::= <i>let</i>
<i>let*</i>	::= <i>let</i> *
<i>letrec</i>	::= <i>letrec</i>
<i>lambda</i>	::= <i>lambda</i>
<i>head</i>	::= <i>head</i>
<i>tail</i>	::= <i>tail</i>
<i>pair</i>	::= <i>pair</i>
<i>cond</i>	::= <i>cond</i>
<i>add1</i>	::= <i>add1</i>
<i>sub1</i>	::= <i>sub1</i>
<i>sqrt</i>	::= <i>sqrt</i>
<i>expt</i>	::= <i>expt</i>
<i>not</i>	::= <i>not</i>
<i>null?</i>	::= <i>null</i> \?

En nuestro lenguaje utilizaremos *null?* para verificar si una lista es vacía o no.

Operadores Artiméticos

+	::= \+
-	::= \-
*	::= *
/	::= \/

Operadores de Comparación

<=	::= <=
>=	::= >=
!=	::= !=
=	::= =
<	::= <
>	::= >

Paréntesis

(+)

Se utilizan para agrupar subexpresiones lógicas. Al igual que en lenguajes de notación matemática o lógica simbólica, permiten definir la precedencia de los operadores.

Delimitadores

```
(      ::= \(
)      ::=\)
[      ::= \[
]      ::= \]
,      ::= ,
ws     ::= [ \t\n\r]+
```

Estos últimos deben ser ignorados por el lexer.

Colocamos entre paréntesis el símbolo de la "," para indicar que también pertenece al alfabeto de nuestro lenguaje LAMBDAE.

En este punto, los tokens del lenguaje representan las unidades léxicas construidas a partir del alfabeto definido. Para esta versión del lenguaje, se incluyen los siguientes tipos de tokens, junto con las expresiones regulares que especifican la estructura de cada uno a partir de la gramática presentada anteriormente.

Tokens en LAMBDAE

int	{ TokenNum \$\$ }
bool	{ TokenBool \$\$ }
var	{ TokenVar \$\$ }
'+'	{ TokenSuma }
'-'	{ TokenResta }
'*'	{ TokenMult }
'/'	{ TokenDiv }
"sqrt"	{ TokenSqrt }
"expt"	{ TokenExpt }
"not"	{ TokenNot }
"null?"	{ TokenNull }
'('	{ TokenPA }
')'	{ TokenPC }
'['	{ TokenCA }
']'	{ TokenCC }
','	{ TokenComa }
'='	{ TokenEq }
'<'	{ TokenMenor }
'>'	{ TokenMayor }
'<='	{ TokenMenorIgual }
'>='	{ TokenMayorIgual }
'!='	{ TokenDistinto }
"if"	{ TokenIf }

"fst"	{ TokenFst }
"snd"	{ TokenSnd }
"let"	{ TokenLet }
"let*"	{ TokenLetStar }
"letrec"	{ TokenLetRec }
"lambda"	{ TokenLambda }
"head"	{ TokenHead }
"tail"	{ TokenTail }
"pair"	{ TokenPair }
"cond"	{ TokenCond }
"add1"	{ TokenAdd1 }
"sub1"	{ TokenSub1 }

3.2. Gramática Formal: gramática libre de contexto (GLC)

En el ámbito de la teoría de los lenguajes de programación y de los lenguajes formales, la sintaxis concreta hace referencia a la estructura específica de un lenguaje de programación, la cual determina la forma exacta en que deben escribirse los programas. Desde un punto de vista matemático, dicha estructura se describe mediante una gramática libre de contexto, la cual se define de la siguiente manera:

Definición: Gramática libre de contexto

Formalmente, una **Gramática Libre de Contexto (GLC)** se define como una tupla

$$G = (N, \Sigma, P, S),$$

donde:

- N es un conjunto finito de símbolos no terminales. Estos representan categorías sintácticas intermedias y permiten construir la jerarquía estructural del lenguaje.
- Σ es un conjunto finito de símbolos terminales, que corresponden a los tokens generados por el análisis léxico.
- P es un conjunto finito de producciones de la forma $A \rightarrow \alpha$, donde $A \in N$ y $\alpha \in (N \cup \Sigma)^*$.
- $S \in N$ es el símbolo inicial, desde el cual se derivan todas las cadenas válidas del lenguaje. [5, 19, 20]

Cada uno de estos componentes tiene un papel específico en la descripción de la sintaxis:

1. **Terminales:** representan los elementos básicos del lenguaje que no pueden ser descompuestos más allá dentro de la gramática. En nuestro caso los terminales son los valores booleanos (*bool*), como valores booleanos resultantes de evaluar operaciones lógicas (*not*), enteros (*int*), pares ordenados (*pair*) y clousures (*Clousure*).

2. **No terminales:** estos corresponden a variables sintácticas que describen cómo se organizan y combinan los diferentes componentes del lenguaje. En nuestra gramática, los no terminales principales serán expresiones que se pueden generar a partir de los elementos definidos en el alfabeto de nuestra gramática, pueden ser binarias y variádicos como lo son: (*Add, Sub, Mul, Div, ..*), funciones (*App, Fun, lambda, ..*), condicionales (*if*), operaciones sobre pares ordenados (*Fst, Snd*), listas (*head, tail*), aplicaciones *let*, comparaciones, entre otras.
3. **Símbolo inicial:** representa el elemento desde el cual se generan todas las expresiones válidas del lenguaje.
4. **Producciones:** son las reglas mediante las cuales se obtienen expresiones más complejas a partir de otras más simples. Estas permiten describir de forma recursiva cómo se combinan operadores, constantes y subexpresiones, dando origen a la sintaxis del lenguaje.

Corrección: GLC de LAMBDAE MÓDULO A2

```
< expr > ::= <consB>
           | <consE>
           | <consV>
           | ( not < expr > )
           | ( < binop > < expr > <expr>+ )
           | ( <unop> <expr> )

<binop> ::= +
         | -
         | *
         | /
         | =
         | <
         | >
         | <=
         | >=
         | pair

<unop> ::= not
        | null?
        | sqrt
        | expt
        | fst
        | snd
        | head
        | tail
        | cond
        | add1
        | sub1

<consB> ::= True
         | False

<consE> ::= 0
         | <digito-no-cero> <digitos>
         | <digito-no-cero>

<digito-no-cero> ::= 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

<digitos> ::= <digito> <digitos>
           | <digito>

<digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

<consV> ::= <letra> <resto-id>
         | <letra>
```

```
<resto-id> ::= <character-id> <resto-id>
              | <character-id>
```

```
<character-id> ::= <letra>
                  | <digito>
                  | _
```

```
<letra> ::= a | b | c | d | e | f | g | h | i | j | k | l | m
          | n | o | p | q | r | s | t | u | v | w | x | y | z
          | A | B | C | D | E | F | G | H | I | J | K | L | M
          | N | O | P | Q | R | S | T | U | V | W | X | Y | Z
```

Utilizamos la construcción *unop* para expresiones que tienen un solo argumento y *binop* para las que pueden tener dos o más argumentos.

Construcciones especiales:

```
⟨lambda⟩ ::= lambda
⟨condicional⟩ ::= if
⟨let⟩ ::= let
⟨let*⟩ ::= let*
⟨letRec⟩ ::= letRec
⟨lista⟩ ::= [ ]
⟨aplicacion⟩ ::= app
```

Es importante resaltar que en nuestro lenguaje LAMBDÆ utilizamos notación prefija, las operaciones se evalúan de forma binaria a pesar de que se puedan tomar de forma n-aria.

Clasificación de Construcciones

- **Operadores unarios:** Operaciones que reciben exactamente un argumento: `not`, `sqrt`, `fst`, `snd`, `head`, `tail`, `add1`, `sub1`, `null?`
- **Operadores binarios:** Operaciones que reciben dos o más argumentos (con asociatividad izquierda para múltiples argumentos): `+`, `-`, `*`, `/`, `=`, `<`, `>`, `<=`, `>=`, `!=`, `pair`
- **Abstracciones (lambda):** Construcción que crea funciones anónimas. Recibe un parámetro (variable) y un cuerpo (expresión): `(lambda <var><expr>)`
- **Condicionales:** Construcción de control de flujo que evalúa una condición y selecciona entre dos ramas: `(if <cond><then><else>)`
- **Vinculaciones locales (let):** Construcciones que introducen variables locales en un ámbito:
 - `let`: Vinculación simple no recursiva
 - `letRec`: Vinculación recursiva para definir funciones recursivas

- **Listas:** Construcción de estructuras de datos secuenciales mediante corchetes: [e1 e2 ... en]
- **Aplicación de funciones:** Construcción que aplica una función (que puede ser una lambda, variable, o expresión que evalúa a función) a uno o más argumentos: (f arg1 arg2 ... argn)

Notación

- $\langle expr \rangle^*$ indica cero o más repeticiones de $\langle expr \rangle$
- $\langle expr \rangle^+$ indica una o más repeticiones de $\langle expr \rangle$, el cual lo utilizamos al definir *binop*.

Para definir la gramática formal utilizamos una gramática libre de contexto que siga la Forma Normal Extendida de Backus-Naur(en inglés Extended Backus–Naur Form o EBNF) que es una notación formal que se utiliza para describir la gramática de un lenguaje de programación (o cualquier lenguaje formal). Para esto comencemos dando un breve contexto.

Notación BNF

Antes de que tuvieramos la notación EBNF, se tenía la BNF(Backus-Naur Form) propuesta por John W. Backus en 1959, la cual surge ante la necesidad de tener una notación formal para describir la sintaxis de lenguajes de programación. Una de sus características principales es su capacidad de definir alternativas mediante el uso del operador |.

La forma BNF introdujo una forma clara y formal de especificar la sintaxis de un lenguaje (producciones, no terminales, terminales, alternancias). Esto permitió que las especificaciones fueran menos ambiguas y más aptas para que herramientas automáticas (analizadores sintácticos / parsers) las usaran. [2]

En nuestro caso no haremos tanta énfasis en este tipo de notación pues no la utilizamos para definir nuestra gramática. [2, 6, 8]

Componentes básicos de BNF

En su forma básica, una regla en BNF se representa mediante una producción con el símbolo ::= y puede incluir alternativas mediante el operador |. Las variables (o símbolos no terminales) suelen escribirse entre signos de mayor y menor (<>), mientras que los símbolos terminales (como palabras reservadas u operadores) se escriben tal cual aparecen en el lenguaje. [3–5]

3.2.1. Notación EBNF

En 1970 Niklaus Wirth propuso notaciones y promovió una forma de extender BNF con construcciones para iteración, opcionalidad y agrupamiento, que se conocen comúnmente como EBNF (Extended Backus–Naur Form). El trabajo de Wirth de 1977 es un hito en la unificación/práctica de una notación extendida y legible. [7]

Fue desarrollada con el propósito de ofrecer una forma más concisa, legible y expresiva de describir la sintaxis de lenguajes formales, incorporando mecanismos para representar repeticiones, secuencias opcionales y agrupaciones. Estas capacidades permiten expresar reglas sintácticas complejas de forma más clara, mejorando tanto la comprensión como la implementación de analizadores sintácticos. Formalmente, la notación EBNF introduce los siguientes elementos adicionales: [2, 4]

- **Repetición:** se denota con llaves , señalando que un componente puede aparecer ninguna o múltiples veces.
- **Opcionalidad:** se marca con corchetes [], mostrando elementos que pueden estar presentes o ausentes en una regla.
- **Agrupación:** se implementa usando paréntesis (), facilitando el control de prioridades y la delimitación de opciones.
- **Terminales:** generalmente se escriben entrecomillados para diferenciarlos de los símbolos no terminales.
- **Alternativas:** continúan expresándose mediante el símbolo |, tomado de la notación BNF tradicional. [4-6]

Considerando los elementos anteriores y tomando como guía a MiniLisp , tenemos que la gramática en notación en EBNF de LAMBDAAE es la siguiente:

Corrección: Gramática en EBNF de LAMBDAAE MÓDULO A2

```

<S > := < Expr >
<Expr > ::= <int >
           | <bool>
           | <var>
           | '(' "not" <Expr> ')'
           | '(' '+' <Expr-list> ')'
           | '(' '-' <Expr-list> ')'
           | '(' '/' <Expr-list> ')'
           | '(' '*' <Expr-list> ')'
           | '(' '=' <Expr-list> ')'
           | '(' '<' <Expr-list> ')'
           | '(' '>' <Expr-list> ')'
           | '(' "<=" <Expr-list> ')'
           | '(' ">=" <Expr-list> ')'
           | '(' "!=" <Expr-list> ')'
           | '(' "null?" <expr> ')'
           | '(' "lambda" '(' <var-list> ')' <Expr> ')'
           | '(' "sqrt" <expr> ')'
           | '(' "expt" <exp-list> ')'
           | '(' "pair" <expr> <expr> ')'
           | '(' "fst" <expr> ')'
           | '(' "snd" <expr> ')'
           | '(' <expr> ',' <expr> ')'
           | '(' "if" <expr> <expr> <expr> ')'
           | '(' "let" '(' <bindings> ')' <expr> ""
           | '(' "let*" '(' <bindings> ')' <expr> ')'
           | [{ Expr }]
           | '(' "letrec" '(' <var> <expr> ')' <expr> ')'
           | '[' <list-items> ']'
           | '[' ']'

```

```

    | '(' "head" <expr> ')'
    | '(' "tail" <expr> ')'
    | '(' "cond" <cond-clauses> ')'
    | '(' <expr> <expr-list> ')'
    | '(' "add1" <expr> ')'
    | '(' "sub1" <expr> ')'

<exp-list> ::= <expr> { <expr> }
<expr-list> ::= <expr> { <expr> }

<var-list> ::= <var> { <var> }

<list-items> ::= <expr> [ ',' <list-items> ]

<cond-clauses> ::= <cond-clause> { <cond-clause> }
<cond-clause> ::= '[' <expr> <expr> ']'
                | '[' "else" <expr> ']'

<bindings> ::= <binding> { <binding> }

<binding> ::= '(' <var> <expr> ')''

<int > := <N >
        | -<M >
<D > := 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<N > := 0 | <D >{ <N >}
<M > := <D >{ <N >}

<bool> := <#t, #f>

<var> :=<X>
<X> := <X> ::= a | b | c | d | e | f | g | h | i | j | k | l | m |
        n | o | p | q | r | s | t | u | v | w | x | y | z

```

Estilo de notación tomada de [1,2,11]

Todos los operadores asocian hacia la izquierda, lo que permite omitir paréntesis en muchos casos sin ambigüedad. Estas convenciones simplifican tanto la escritura como el procesamiento de expresiones regulares en analizadores léxicos. [12,14] Por lo anterior podemos definir lo siguiente para las operaciones aritméticas variádicas:

- $(+ e_1 e_2 e_3 \dots e_n)$ se evalúa como $(+ (+ (+ e_1 e_2) e_3) \dots e_n)$
- $(- e_1 e_2 e_3)$ se evalúa como $(- (- e_1 e_2) e_3)$
- $(* e_1 e_2 e_3)$ se evalúa como $(* (* e_1 e_2) e_3)$
- $(/ e_1 e_2 e_3)$ se evalúa como $(/ (/ e_1 e_2) e_3)$

Para los operadores de comparación:

- `(= e1 e2 e3 ... en)` verifica que `e1 = e2 AND e2 = e3 AND ... en-1 = en`
- Similar para `<`, `>`, `<=`, `>=`, `!=`

Listas:

- `[e1 e2 ... en]` construye una lista mediante reducción por cons: `(cons e1 (cons e2 ... (cons en nil)))`

Aplicación de funciones:

- `(f arg1 arg2 ... argn)` aplica la función `f` a los argumentos de izquierda a derecha

Let* secuencial:

- `(let* ((x1 e1) (x2 e2) ... (xn en)) body)` vincula las variables secuencialmente, donde cada expresión `ei` puede usar las variables `x1`, ..., `xi-1` previamente definidas

4. Sintaxis Abstracta

En la sintaxis concreta se especifican de forma minuciosa todos los elementos necesarios para construir las expresiones válidas del lenguaje. No obstante, esta descripción no es suficiente para capturar toda la organización y el comportamiento lógico de un programa. La sintaxis concreta incluye una gran cantidad de detalles, como símbolos de separación, espacios y paréntesis que son indispensables para interpretar el código, pero que no aportan directamente a la comprensión de su estructura conceptual. De hecho, dichos elementos pueden volver más complejo el análisis y la manipulación del código, especialmente cuando se llevan a cabo procesos de optimización durante la traducción.

La sintaxis abstracta a diferencia de la anterior, esta ofrece una representación simplificada y estructural del código fuente, enfocada en capturar la lógica y la jerarquía del programa sin incluir los detalles superficiales propios de la sintaxis concreta. La forma más común de representarla es mediante un Árbol de Sintaxis Abstracta (ASA), el cual facilita tanto el análisis como la manipulación del código en las fases de compilación o interpretación.

Todos estos elementos que nos son relevantes o que no aportan directamente algo a la comprensión conceptual los podemos simplificar utilizando *azúcar sintáctica* la cual se encargará de eliminar estos componentes que no son necesarios y estandarizar todas las expresiones que sean admitidas por nuestro lenguaje a una misma. [3–5]

4.1. Árboles de sintaxis

Los árboles de sintaxis constituyen una herramienta fundamental en la representación estructurada de un programa, ya que permiten visualizar cómo se organizan y combinan los distintos elementos de acuerdo con las reglas del lenguaje. Mediante esta estructura jerárquica, es posible determinar la precedencia de los operadores y comprender de manera precisa cómo deben evaluarse las expresiones.

Una ventaja clave de estos árboles es que ayudan a eliminar posibles ambigüedades al interpretar el código. Si una gramática permite construir más de un árbol sintáctico

para la misma cadena, se generan múltiples significados para un mismo programa, lo cual resulta problemático en un lenguaje de programación. Por ello, los árboles sintácticos sirven como evidencia de la claridad o ambigüedad de la gramática que define al lenguaje.

En los compiladores e intérpretes, estos árboles son el resultado directo de la fase de análisis sintáctico (parsing), en la cual el código fuente se transforma en una representación estructurada y más manejable. Esta representación se convierte en la base sobre la cual se realizan posteriores procesos como el análisis semántico y la traducción a código ejecutable.

Además, cuando se formaliza la sintaxis abstracta de un lenguaje como LAMBDAE, se logra una versión aún más depurada de esta estructura. La sintaxis abstracta prescinde de detalles decorativos de la notación concreta y se enfoca en capturar únicamente la lógica y las relaciones semánticas entre los componentes del programa. Gracias a ello, se facilita el desarrollo de herramientas como analizadores, traductores y optimizadores, que pueden operar sobre una representación más simple, eficiente y extensible.

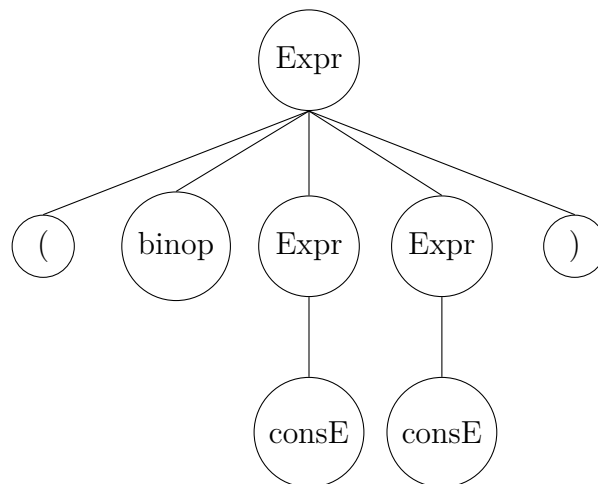
En conjunto, los árboles de sintaxis y la sintaxis abstracta aportan una comprensión conceptual más clara del lenguaje y proporcionan los cimientos necesarios para la construcción de un sistema de compilación sólido, coherente y capaz de evolucionar con el tiempo.

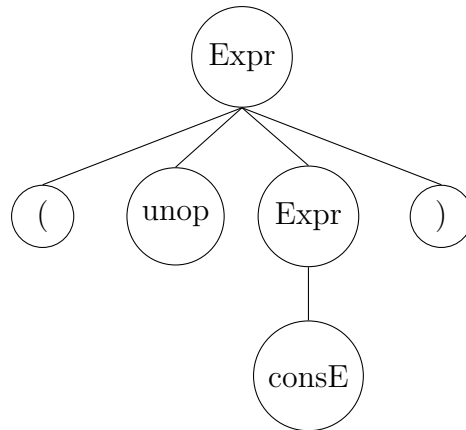
4.1.1. Árboles de sintaxis concreta

En este modelo, un programa puede representarse mediante un ASC que conserva todos los elementos presentes en el código fuente (como operadores, paréntesis y demás detalles sintácticos), reflejando paso a paso la derivación completa definida por la gramática.

Consideremos el siguiente árbol de sintaxis concreta (ASC) del lenguaje LAMBDAE que represneta las expresiones binarias:

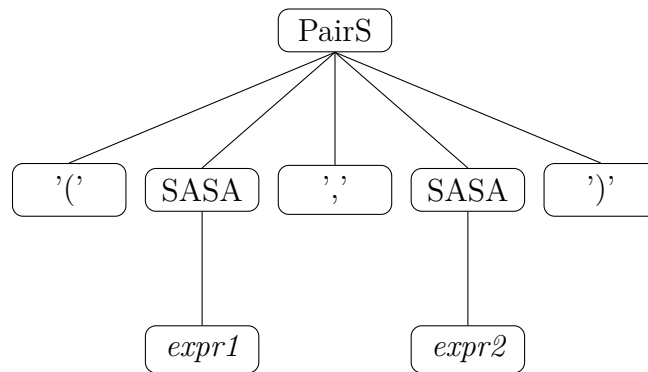
Esta expresión se deriva usando la gramática presentada. El ASC ilustra la derivación de la expresión, mostrando de forma explícita cada símbolo no terminal y terminal (paréntesis, operador y valores):



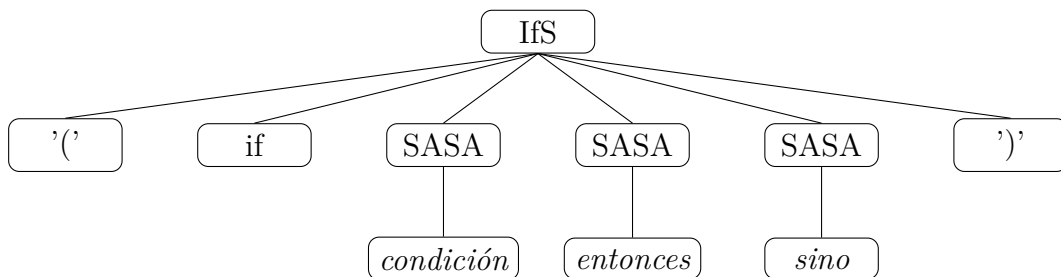


Con este tipo de árbol estamos representando a las expresiones que se pueden generar con *null* , *sqrt* , *expt* , *fst* , *scnd* , *cond* , *head* , *tail* , *add1* , *sub1* , por lo que son análogos. Ahora veamos casos más concretos:

■ **ASC de par con coma:**



■ **ASC de if:**



En los árboles obtenidos podemos observar que:

- La raíz está etiquetada como *expr*, derivando la estructura completa.
- Se incluyen todos los elementos sintácticos, como los paréntesis y operadores.
- Cada nodo refleja una regla aplicada de la gramática, de forma que la estructura del árbol corresponde a la derivación concreta de la expresión.

A partir de estas bases teóricas, procederemos a definir los Árboles de Sintaxis Concreta correspondientes a las distintas construcciones que forman parte de nuestro lenguaje de programación. Para ello, presentaremos ejemplos representativos de expresiones válidas dentro del lenguaje y su respectiva interpretación mediante un ASC. Este enfoque nos permitirá ilustrar cómo las reglas gramaticales se reflejan de forma explícita en la estructura jerárquica del árbol, mostrando con claridad la combinación y precedencia de los elementos sintácticos que intervienen en cada expresión. De esta manera, se establecerá una correspondencia precisa entre la especificación formal del lenguaje y su representación estructurada, lo cual será fundamental para etapas posteriores del análisis y procesamiento del código.

4.1.2. Árboles de sintaxis abstracta (ASA)

Un árbol de sintaxis abstracta (AST) es una estructura de datos de tipo árbol que representa la estructura sintáctica abstracta de un fragmento de código fuente en un lenguaje de programación.

“Abstracta” significa que no incluye todos los detalles del texto fuente tal como estaban escritos, sino sólo los elementos relevantes para la estructura del programa (por ejemplo, se omiten paréntesis redundantes, llaves, puntos y comas cuando no cambian la estructura semántica).

Representa de forma jerárquica los constructores del lenguaje (como expresiones, sentencias, declaraciones) como nodos conectados. Por ejemplo, una operación binaria “a + b” se representa por un nodo “+” con dos hijos: “a” y “b”.

Es el resultado habitual de la fase de análisis sintáctico (parser) en la construcción de compiladores o intérpretes, que transforma el código fuente en una estructura más manejable para el análisis posterior.

Definición: Árboles de sintaxis abstracta

Un árbol de sintaxis abstracta es un árbol ordenado $A = (N, A, R)$ donde:

- N es un conjunto finito de nodos que representan las construcciones del lenguaje mediante etiquetas, y las hojas representan sus respectivos valores.
- $A \subseteq N \times N$ es un conjunto de aristas dirigidas que conectan a los nodos.
- $R \in N$ es la raíz del árbol. [5,6]

Así pues, el AST captura lo esencial de la estructura del programa, dejando de lado detalles de la representación textual que no afectan su significado.

En el caso del lenguaje LAMBDAE, los árboles de sintaxis abstracta serán los que se generarán una vez que quitemos el azúcar sintáctica de todas las expresiones de nuestro lenguaje. Podemos decir que los ASA que tendremos en nuestro lenguaje serán de la siguiente forma:

La conexión entre la sintaxis concreta y la sintaxis abstracta puede describirse de forma rigurosa mediante la composición de los procesos de análisis léxico y análisis sintáctico, ya mencionados previamente en clase. En este proceso, el analizador léxico (*lexer*) recibe como entrada el código fuente y lo convierte en una lista de tokens, mientras que el analizador sintáctico (*parser*) toma dicha lista de tokens y genera una representación estructurada, como lo es un Árbol de Sintaxis Abstracta (ASA).

Descripción formal:

Sea S una cadena de texto proveniente del programa fuente. El *lexer* se encarga de producir una lista de tokens $[Token]$ aplicando una función de transformación definida como:

$$S \rightarrow [Token].$$

Cada token corresponde a una unidad léxica relevante del lenguaje: identificadores, operadores, palabras clave, entre otros.

A continuación, el *parser* utiliza esa lista $[Token]$ para construir una estructura sintáctica jerárquica, típicamente un ASA, mediante una función definida como:

$$[Token] \rightarrow ASA.$$

En consecuencia, la relación entre la sintaxis concreta y la sintaxis abstracta, que denotamos mediante φ , se define como la composición de ambas funciones: [3, 6, 22]

$$\varphi = \text{parser} \circ \text{lexer} : S \rightarrow ASA.$$

Corrección: ASA de LAMBDAAE MÓDULO A3

Representación por reglas

Un árbol de sintaxis abstracta se puede interpretar como una estructura arbórea ordenada donde cada punto del árbol lleva la etiqueta de un operador específico. Cada uno de estos operadores tiene asociado un valor numérico que especifica cuántos argumentos puede recibir, y este valor coincide exactamente con la cantidad de nodos descendientes que deben existir en los puntos donde se emplea ese operador. Con base en estos conceptos, se establece la relación eASA, que señala que e representa un árbol de sintaxis abstracta válido. La construcción de estos árboles se especifica mediante las siguientes reglas: [3–5]

Tenemos entonces los siguientes tipos de expresiones:

- **Enteros**, los cuales representaremos con la etiqueta **Num**. Al ser un valor atómico, únicamente tendrán como hijo al valor entero que representan.
- **Booleanos**, los representaremos con la etiqueta **Bool**. Al ser un valor atómico, únicamente tendrán como hijo al valor booleano que representan.
- **Variables**, las cuales etiquetaremos con la etiqueta **Id**. Al ser un valor atómico, únicamente tendrán como hijo al valor de una variable que representan.
- **Operaciones Binarias**, las cuales etiquetaremos con la etiqueta **BinOp**. Como podemos apreciar en la gramática, las operaciones aritméticas y las comparaciones requieren dos expresiones, por lo que se trata de un nodo con dos hijos.
- **Raíz cuadrada**, las cuales etiquetaremos con la etiqueta **Sqrt**. Se tratará de un nodo que tengan un solo hijo.
- **Negación**, las cuales etiquetaremos con la etiqueta **Not**. Al igual que la raíz cuadrada, se tratará de un nodo con un solo hijo.

- **Operaciones Unarias**, las cuales etiquetaremos con la etiqueta **UnOp**. Se tratará de nodos que tengan un solo hijo.
- **Pares**, las cuales etiquetaremos con la etiqueta **Pair**. Se tratará de nodos que tengan dos hijos.
- **Primer elemento de un par ordenado**, las cuales etiquetaremos con la etiqueta **Pair**. Se tratará de nodos que tengan dos hijos.
- **Segundo elemento de un par ordenado**, las cuales etiquetaremos con la etiqueta **Snd**. Se tratará de nodos que tengan dos hijos.
- **If**, las cuales etiquetaremos con la etiqueta **If**. Se tratará de nodos que tengan dos o tres hijos.
- **Funciones**, las cuales etiquetaremos con la etiqueta **Fun**. Al igual que las operaciones binarias, se tratará de nodos que tengan dos hijos.
- **Aplicaciones de función**, las cuales etiquetaremos con la etiqueta **App**. Se tratará de nodos que tengan dos hijos.
- **Closure**, las cuales etiquetaremos con la etiqueta **Clousure**.

Para definir las reglas que nos permitan especificar a los ASA, definimos la siguiente relación:

$$a \in \text{ASA}$$

que se lee como: a es un ASA.

Procedemos a definir las reglas y colocamos una descripción de cuál sería su lectura. [3,5]

Enteros

Num(n) es un ASA si n es un número entero. Es un nodo hoja que contiene el valor numérico.

$$\frac{n \in \mathbb{Z}}{\text{Num}(n) \in \text{ASA}}$$

Booleanos

Boolean b es un ASA si b es un valor booleano. Es un valor terminal.

$$\frac{b \in \{\text{True}, \text{False}\}}{\text{Boolean}(b) \in \text{ASA}}$$

Varibales

Id x es un ASA si x es una cadena que representa un identificador válido. Es un valor terminal que representa variables.

$$\frac{x \in \text{String}}{\text{Id}(x) \in \text{ASA}}$$

Operaciones Binarias

$\text{BinOp } op \ e_1 \ e_2$ es un ASA si op es un operador válido y tanto e_1 como e_2 son ASAs. Es un nodo con dos subárboles que representa operaciones binarias.

Operadores válidos en Op :

- **Aritméticos:** Add, Sub, Mul, Div
- **Comparación:** Eq, Menor, Mayor, MenorIgual, MayorIgual, Distinto

$$\frac{Op \in Op \quad e_1 \in ASA \quad e_2 \in ASA}{\text{BinOp}(e_1, e_2) \in ASA}$$

Raíz Cuadrada

$\text{Sqrt } e$ es un ASA si e es un ASA. Representa la operación de raíz cuadrada. Es un nodo con un subárbol.

$$\frac{e \in ASA}{\text{Sqrt } e \in ASA}$$

Negación

$\text{Not } e$ es un ASA si e es un ASA. Representa la negación lógica. Es un nodo con un subárbol.

$$\frac{e \in ASA}{\text{Not } e \in ASA}$$

Operaciones Unarias $\text{UnOp } op \ e$ es un ASA si op es un operador unario válido y e es un ASA. Representa operaciones unarias genéricas.

Operadores unarios válidos:

- NullOp – verificar lista vacía
- AddOp – incrementar en 1
- SubOp – decrementar en 1
- HeadOp – cabeza de lista
- TailOp – cola de lista

$$\frac{op \in Op \quad e \in ASA}{\text{UnOp}(op, e) \in ASA}$$

Pares

$\text{Pair } e_1 \ e_2$ es un ASA si tanto e_1 como e_2 son ASAs. Representa un par ordenado (una tupla de dos elementos). Es un nodo con

$$\frac{e_1 \in ASA \quad e_2 \in ASA}{Pair(e_1, e_2) \in ASA}$$

Primer elemento de un par ordenado

Fst e es un ASA si $e \in ASA$. Representa la extracción del primer elemento de un par. Es un nodo con un subárbol.

$$\frac{e \in ASA}{Fst(e) \in ASA}$$

Segundo elemento de un par

Snd e es un ASA si $e \in ASA$. Extrae el segundo elemento de un par. Es un nodo con un subárbol.

$$\frac{e \in ASA}{Snd(e) \in ASA}$$

Condicional If

If $cond \ e_t \ e_e$ es un ASA si $cond$, e_t y e_e son ASAs. Representa una expresión condicional con tres subárboles: condición, rama *then* y rama *else*.

$$\frac{cond \in ASA \quad e_t \in ASA \quad e_e \in ASA}{If(cond, e_t, e_e) \in ASA}$$

Funciones

Fun $param \ body$ es un ASA si $param$ es una cadena que representa el parámetro formal y $body$ es un ASA que representa el cuerpo de la función. Esta construcción corresponde a la forma núcleo de una lambda unaria (con un solo parámetro).

$$\frac{param \in String \quad body \in ASA}{Fun(param, body) \in ASA}$$

Las funciones con múltiples parámetros se representan mediante *currying*, es decir, lambdas anidadas. Por ejemplo, en Haskell:

```
Fun "x" (Fun "y" (BinOp Add (Id "x") (Id "y")))
```

Representa la expresión lambda:

$$(\lambda x. (\lambda y. x + y))$$

Aplicaciones de función (binaria)

Descripción: **App** $func \ arg$ es un ASA si tanto $func$ como arg son ASAs. Representa la aplicación de una función a un argumento. Esta es la forma núcleo de aplicación binaria (un solo argumento).

$$\frac{func \in ASA \quad arg \in ASA}{App(func, arg) \in ASA}$$

Las aplicaciones con múltiples argumentos se representan mediante aplicaciones anidadas (currying):

```
haskellApp (App (Id "suma") (Num 3)) (Num 5)
```

Esto representa matemáticamente:

$$(\text{suma } 3) \ 5$$

Closure

Closure *param body env* es un ASA si:

- *param* es el parámetro formal (una cadena),
- *body* es el cuerpo de la función (un ASA),
- *env* es una lista de asociaciones (entorno capturado) donde cada par (x, v) enlaza un identificador x con un valor v .

$$\frac{param \in \text{String} \quad body \in \text{ASA} \quad env \in [(String, \text{ASA})]}{Closure(param, body, env) \in \text{ASA}}$$

Una closure no aparece en el código fuente, sino que es creado durante la evaluación cuando se encuentra una expresión **Fun**. La clausura captura el entorno léxico en el momento de su creación.

$$\frac{e \in \text{ASA} \quad env \in [(String, \text{ASA})]}{Expr(e, env) \in \text{ASA}}$$

Expr *e env* es un ASA si:

- *e* es la expresión a evaluar (*ASA*),
- *env* es una lista de asociaciones que representa el entorno de evaluación.

Expr no aparece en el código fuente. Es una construcción semántica utilizada durante la evaluación para representar la evaluación de una expresión en un entorno específico.

Contexto de uso: Se utiliza principalmente en la regla de aplicación de funciones (paso pequeño), donde es necesario evaluar el cuerpo de una función en el entorno capturado por la clausura, extendido con el enlace del parámetro al argumento.

5. Azúcar sintáctica

El azúcar sintáctica es un concepto fundamental en el diseño de lenguajes de programación que data de 1964, cuando el científico de la computación Peter J. Landin acuñó este término para describir la sintaxis superficial de un lenguaje similar a ALGOL [17, 23]. Originalmente, Landin utilizó este concepto para referirse a características sintácticas que podían ser definidas semánticamente en términos de expresiones lambda más simples.

Con el tiempo, lenguajes como CLU, ML y Scheme extendieron el término para referirse a cualquier sintaxis dentro de un lenguaje que pudiera ser definida en términos de un núcleo de construcciones esenciales.

Definición: Azúcar Sintáctica

En ciencias de la computación, el azúcar sintáctica se refiere a la sintaxis dentro de un lenguaje de programación que está diseñada para hacer las cosas más fáciles de leer o expresar [14, 15, 23].

Esta hace que el lenguaje sea “más dulce” para el uso humano: las cosas pueden expresarse con mayor claridad, de manera más concisa, o en un estilo alternativo que algunos programadores pueden preferir.

De manera más formal, el azúcar sintáctica se define como aquellas características opcionales en los lenguajes de programación que proporcionan múltiples formas de realizar tareas similares [4].

Son construcciones sintácticas cuyos efectos están definidos en términos de otras construcciones sintácticas ya existentes en el lenguaje. Los procesadores de lenguaje, incluyendo compiladores y analizadores estáticos, frecuentemente expanden estas construcciones “azucaradas” en sus equivalentes más verbosos antes de procesarlas, un proceso denominado “desazucarado” o *desugaring*.

El azúcar sintáctica desempeña un papel crucial en el desarrollo moderno de software por diversas razones fundamentales:

Mejora de la legibilidad y expresividad. El azúcar sintáctica permite que el código se escriba de manera más natural e intuitiva, facilitando que los programadores comprendan rápidamente el propósito y el flujo del código [14,23]. Esto es particularmente valioso en proyectos grandes donde múltiples desarrolladores deben colaborar y entender código escrito por otros.

Reducción del código repetitivo. Al proporcionar formas más concisas de expresar operaciones comunes, el azúcar sintáctica elimina la necesidad de código repetitivo (*boilerplate*) y expresiones idiomáticas difíciles de descifrar. Esto no solo ahorra tiempo de desarrollo, sino que también reduce las oportunidades de introducir errores.

Abstracción de alto nivel. El azúcar sintáctica permite a los programadores trabajar en un nivel de abstracción más elevado, enfocándose en la lógica del problema en lugar de los detalles de implementación de bajo nivel. Esto facilita la escritura de código que se asemeja más al lenguaje del dominio del problema que al lenguaje de la máquina [19, 23].

Simplificación de la implementación del compilador. Al implementar características del lenguaje en el *frontend* mediante desazucarado, se reduce la complejidad del *backend* del compilador, ya que no es necesario agregar nuevos tipos de nodos al árbol de sintaxis abstracta (AST) para cada nueva característica sintáctica.

Sin embargo, es importante reconocer que el azúcar sintáctica también presenta desafíos. La inclusión de múltiples características opcionales aumenta la complejidad de la especificación del lenguaje y puede causar problemas a medida que los programas crecen en tamaño y complejidad [23]. Algunos programadores, particularmente en la comunidad Lisp, consideran que estas características de usabilidad sintáctica son innecesarias o incluso frívolas, como lo expresó Alan Perlis en su famosa frase: “El azúcar sintáctica causa cáncer de los punto y coma”.

A pesar de estas críticas, el azúcar sintáctica continúa siendo una herramienta valiosa en el diseño de lenguajes de programación modernos, equilibrando la necesidad de expresividad y facilidad de uso con los requisitos de simplicidad y uniformidad del lenguaje.

En el caso de LAMBDAE pasaremos de tener los SASA que son los definidos en la gramática, a tener los ASA.

5.1. Eliminación de azúcar sintáctica: el Proceso de Desugaring en LAMBDAE

Justificación Teórica del Desugaring

El proceso de desugaring es una técnica fundamental en el diseño e implementación de lenguajes de programación modernos. Como establece la literatura, los procesadores de lenguaje, incluyendo compiladores y analizadores estáticos, frecuentemente expanden las construcciones azucaradas en sus equivalentes más verbosos antes de procesarlas [14, 15]. Este enfoque arquitectónico permite separar claramente la sintaxis superficial del lenguaje de su semántica esencial.

El desugaring es esencialmente una transformación de sintaxis a sintaxis: una función que mapea expresiones escritas en una sintaxis rica y conveniente hacia un conjunto reducido de construcciones primitivas [14]. Esta estrategia ofrece beneficios arquitectónicos significativos: al reducir el número de construcciones que deben ser procesadas por las etapas posteriores del compilador (evaluación, análisis de tipos, optimización), se simplifica dramáticamente la implementación de estos componentes [15, 24].

En el contexto de nuestro lenguaje LAMBDAE, el proceso de desugaring se manifiesta como una transformación explícita entre dos representaciones de árbol sintáctico abstracto (AST): **SASA** (Syntactic Abstract Syntax tree with Sugar Added) y **ASA** (Abstract Syntax tree After desugaring). Esta distinción arquitectónica refleja el principio de que las características sintácticas convenientes deben ser eliminadas antes del procesamiento semántico.

SASA y ASA

La arquitectura de LAMBDAE implementa una separación clara entre la sintaxis superficial (SASA) y el lenguaje núcleo (ASA). Esta separación es consistente con las prácticas establecidas en el diseño de lenguajes funcionales, donde el azúcar sintáctica se define en términos de un núcleo de construcciones esenciales.

5.1.1. SASA: Sintaxis Rica con Azúcar Sintáctica

El árbol SASA es producido directamente por el parser y contiene todas las construcciones sintácticas convenientes que el lenguaje ofrece al programador. Estas construcciones incluyen:

1. **Operaciones n-arias:** Expresiones como $(+ \ e1 \ e2 \ \dots \ en)$ permiten operaciones sobre múltiples argumentos, eliminando la necesidad de anidar operaciones binarias.
2. **Funciones lambda con múltiples parámetros:** La sintaxis $(\text{lambda } (x \ y \ z) \ \text{body})$ es más natural que las funciones unarias anidadas.

3. **Construcciones let especializadas:** LAMBDAE distingue entre `let`, `let*` y `letrec`, cada una con semántica diferente para el alcance de variables.
4. **Azúcar sintáctica de conveniencia:** Expresiones como `(add1 e)` y `(sub1 e)` que son abreviaciones de operaciones más complejas.
5. **Expresiones condicionales múltiples:** La construcción `cond` que maneja múltiples casos de manera más elegante que `if` anidados.

Como observa Rathi en su análisis de compiladores, estas abstracciones sintácticas “hacen la programación más dulce” al permitir código más legible y conciso, pero complican las etapas posteriores del compilador si no se eliminan [14].

Corrección: Reglas de Inferencia SASA

A continuación se presentan las **reglas formales del árbol sintáctico SASA**, expresadas mediante sistemas de inferencia. Estas reglas especifican cómo cada construcción del lenguaje es reconocida y estructurada por el parser dentro del árbol SASA, asignando una forma canónica a expresiones que, aunque puedan variar en su superficie sintáctica, comparten una misma interpretación semántica. El objetivo de estas reglas es capturar de manera precisa la correspondencia entre la sintaxis concreta del lenguaje y su representación abstracta, facilitando posteriormente las fases de desazucaramiento, análisis semántico y traducción a la representación AL. Las reglas se presentan usando el estilo de derivación típico en la literatura formal, donde cada constructor del lenguaje queda caracterizado como un nodo válido del árbol SASA.

$$\frac{n \in int}{n \in SASA}$$

$$\frac{b \in bool}{b \in SASA}$$

$$\frac{x \in var}{x \in SASA}$$

$$\frac{e \in SASA}{(not\ e) \in SASA}$$

$$\frac{e \in SASA}{(null?\ e) \in SASA}$$

$$\frac{e \in SASA}{(sqrt\ e) \in SASA}$$

$$\frac{e \in SASA}{(fst\ e) \in SASA}$$

$$\frac{e \in SASA}{(snd\ e) \in SASA}$$

$$\frac{e \in SASA}{(add1\ e) \in SASA}$$

$$\begin{array}{c}
\frac{e \in SASA}{(sub1\ e) \in SASA} \\
\\
\frac{e_1, \dots, e_n \in SASA}{(+\ e_1 \ \dots \ e_n) \in SASA} \\
\\
\frac{e_1, \dots, e_n \in SASA}{(-\ e_1 \ \dots \ e_n) \in SASA} \\
\\
\frac{e_1, \dots, e_n \in SASA}{(*\ e_1 \ \dots \ e_n) \in SASA} \\
\\
\frac{e_1, \dots, e_n \in SASA}{(/ \ e_1 \ \dots \ e_n) \in SASA} \\
\\
\frac{e_1, \dots, e_n \in SASA}{(=\ e_1 \ \dots \ e_n) \in SASA} \\
\\
\frac{e_1 \in SASA \quad e_2 \in SASA}{(pair\ e_1\ e_2) \in SASA} \\
\\
\frac{e_1 \in SASA \quad e_2 \in SASA}{(e_1, \ e_2) \in SASA} \\
\\
\frac{c \in SASA \quad t \in SASA \quad f \in SASA}{(if\ c\ t\ f) \in SASA} \\
\\
\frac{(x_1, e_1), \dots, (x_n, e_n) \text{ son bindings} \quad e_{body} \in SASA}{(let\ ((x_1\ e_1) \dots (x_n\ e_n))\ e_{body}) \in SASA} \\
\\
\frac{(x_1, e_1), \dots, (x_n, e_n) \quad e_{body} \in SASA}{(let^*\ ((x_1\ e_1) \dots (x_n\ e_n))\ e_{body}) \in SASA} \\
\\
\frac{x \in var \quad e_1 \in SASA \quad e_2 \in SASA}{(letrec\ (x\ e_1)\ e_2) \in SASA} \\
\\
\frac{x_1, \dots, x_n \in var \quad e \in SASA}{(lambda\ (x_1 \dots x_n)\ e) \in SASA} \\
\\
\frac{e_1, \dots, e_n \in SASA}{[e_1, \dots, e_n] \in SASA} \\
\\
\frac{}{[] \in SASA} \\
\\
\frac{e \in SASA}{(head\ e) \in SASA} \\
\\
\frac{e \in SASA}{(tail\ e) \in SASA} \\
\\
\frac{[c_1\ e_1], \dots, [c_n\ e_n] \text{ cláusulas válidas}}{(cond\ [c_1\ e_1] \dots [c_n\ e_n]) \in SASA} \\
\\
\frac{f \in SASA \quad a_1, \dots, a_n \in SASA}{(f\ a_1 \ \dots \ a_n) \in SASA}
\end{array}$$

5.1.2. ASA: Lenguaje Núcleo Desugared

El árbol ASA representa el lenguaje núcleo de LAMBDAE, conteniendo únicamente las construcciones primitivas necesarias para la evaluación. Este núcleo se caracteriza por:

1. **Operaciones binarias exclusivamente:** Todas las operaciones n-arias se reducen a composiciones de operaciones binarias mediante `foldl1` o `foldr1`.
2. **Funciones lambda unarias:** Toda función lambda acepta exactamente un parámetro. Las funciones multi-parámetro se representan mediante *currying*.
3. **Aplicación de función binaria:** La aplicación de función es estrictamente binaria, siguiendo el modelo del cálculo lambda.
4. **Eliminación de construcciones derivadas:** Expresiones como `let`, `let*` y `cond` se traducen a combinaciones de `lambda`, aplicación de función, e `if`.

Esta reducción a un núcleo mínimo es consistente con el principio de que “el azúcar sintáctica se puede definir en términos de un conjunto núcleo de construcciones esenciales” [14].

5.2. Transformaciones de Desugaring en LAMBDAE

El módulo `Desugar` de LAMBDAE implementa la función central `desugar :: SASA -> ASA` que realiza la transformación sintáctica. A continuación, analizamos las transformaciones más significativas:

5.2.1. Desugaring de Operaciones N-arias

Las operaciones aritméticas y relacionales que aceptan múltiples argumentos se transforman en cadenas de operaciones binarias:

Listing 1: Desugaring de operaciones n-arias

```
desugar (AddListS es) = foldl1 (BinOp AddOp) (map desugar es)
desugar (SubListS es) = foldl1 (BinOp SubOp) (map desugar es)
desugar (MulListS es) = foldl1 (BinOp MulOp) (map desugar es)
```

Por ejemplo, la expresión SASA `(+ 1 2 3 4)` se desugarea a:

```
BinOp AddOp (BinOp AddOp (BinOp AddOp (Num 1) (Num 2)) (Num 3)) (Num 4)
```

Esta transformación refleja el principio de que operaciones complejas pueden expresarse mediante composición de operaciones más simples [12].

5.2.2. Desugaring de Funciones Lambda Multi-parámetro

Las funciones lambda con múltiples parámetros se transforman en funciones unarias anidadas mediante currying:

Listing 2: Desugaring de funciones lambda

```
desugar (FunListS [] body)      = desugar body
desugar (FunListS (x:xs) body) = Fun x (desugar (FunListS xs body))
```

Una expresión SASA como `(lambda (x y z) (+ x y z))` se convierte en:

```
Fun "x"(Fun z"(Fun "z"(BinOp AddOp (BinOp AddOp (Id "x") (Id z")) (Id "z")))) (Id "z"))))
```

Esta transformación implementa el concepto clásico de currying, donde funciones multi-argumento se representan como secuencias de funciones unarias [12, 14].

5.2.3. Desugaring de Construcciones Let

El desugaring de las construcciones `let` demuestra uno de los principios más elegantes del diseño de lenguajes funcionales: `let` es simplemente azúcar sintáctica para la aplicación inmediata de una función lambda.

Listing 3: Desugaring de let

```
desugar (LetS [(x,e)] body) = App (Fun x (desugar body)) (desugar e)
```

Esta transformación implementa la equivalencia:

$$(\text{let } ((x\ e))\ \text{body}) \equiv ((\text{lambda } (x)\ \text{body})\ e)$$

Esta es una de las transformaciones de desugaring más fundamentales en lenguajes funcionales, demostrando que el enlace de variables puede expresarse completamente en términos de abstracción y aplicación [14, 15].

5.2.4. Desugaring de Let*

La construcción `let*`, que permite referencias secuenciales entre bindings, se desugarea recursivamente en `let` anidados:

Listing 4: Desugaring de let*

```
desugar (LetStarS [] body) = desugar body
desugar (LetStarS ((x,e):rest) body) =
desugar (LetS [(x,e)] (LetStarS rest body))
```

Esta transformación ilustra cómo el azúcar sintáctica puede construirse en capas: `let*` se reduce a `let`, que a su vez se reduce a `lambda` y aplicación.

5.2.5. Desugaring de Expresiones Condicionales Múltiples

La construcción `cond` se desugarea recursivamente en expresiones `if` anidadas:

Listing 5: Desugaring de cond

```
desugarCond :: [(SASA, SASA)] -> ASA
desugarCond [] = error "cond sin clausulas"
desugarCond [(g,e)] = desugar e
desugarCond ((g,e):rest) =
  If (desugar g) (desugar e) (desugarCond rest)
```

Una expresión como:

```
(cond
  [(< x 0) "negativo"]
  [(= x 0) "cero"]
  [else "positivo"])
```

Se transforma en:

```
If (BinOp LtOp x 0) "negativo"(If (BinOp EqOp x 0) "zeropositivo")
```

5.2.6. Desugaring de Azúcar de Conveniencia

LAMBDAE incluye construcciones de conveniencia como `add1` y `sub1` que se desugarean directamente en el parser:

Listing 6: Azúcar sintáctica de conveniencia

```
| '( ' "add1" SASA ' ) ' { AddListS [ $3 , NumS 1 ] }  
| '( ' "sub1" SASA ' ) ' { SubListS [ $3 , NumS 1 ] }
```

Estas expresiones ilustran el principio de que incluso operaciones simples pueden beneficiarse del azúcar sintáctica para mejorar la legibilidad del código.

Corrección: Desugar modulo A4

Proceso de Desazucaramiento en LAMBDAE

Introducción al Desazucaramiento

El *desazucaramiento* (*desugaring*) es el proceso de traducción desde una sintaxis superficial rica (SASA – *Surface Abstract Syntax*) hacia una sintaxis núcleo minimal (ASA – *Abstract Syntax*). Este proceso es fundamental en el diseño de LAMBDAE, ya que permite ofrecer al programador construcciones expresivas y convenientes, mientras se mantiene un núcleo semántico simple y bien definido.

LAMBDAE es un lenguaje que extiende `MiniLisp` con las siguientes características clave:

- **Alcance estático (léxico):** Las variables se resuelven según el entorno donde fueron definidas, no donde fueron invocadas.
- **Estrategia de evaluación ansiosa** (*eager/call-by-value*): Los argumentos se evalúan completamente antes de aplicar funciones.
- **Notación prefija:** Los operadores preceden a sus operandos, siguiendo la tradición de Lisp.

1. Arquitectura del Sistema de Desazucaramiento

1.1 Separación de Sintaxis

El diseño del compilador separa claramente dos niveles sintácticos:

SASA (Surface Abstract Syntax)

- Generada directamente por el parser (*Happy*).
- Contiene toda la riqueza sintáctica del lenguaje.

- Incluye construcciones convenientes como operadores variádicos, `let*`, `letrec`, `cond`, listas con sintaxis `[]`.

ASA (Abstract Syntax)

- Representación núcleo minimal.
- Solo dos formas de función:
 - lambda unaria: `Fun`
 - aplicación binaria: `App`
- Operadores convertidos a forma binaria mediante `BinOp`.
- Base para la evaluación mediante la semántica operacional.

-- Sintaxis núcleo minimal

data ASA

```
= Num Int | Boolean Bool | Id String
| BinOp Op ASA ASA      -- Operadores binarios
| Fun String ASA        -- Lambda UNARIA (núcleo)
| App ASA ASA           -- Aplicación BINARIA (núcleo)
| If ASA ASA ASA
| Pair ASA ASA | Fst ASA | Snd ASA
| Sqrt ASA | Not ASA | UnOp Op ASA
| Closure String ASA [(String, ASA)] -- Runtime
| Expr ASA [(String, ASA)]           -- Runtime
```

La estructura de desazucaramiento se fundamenta en la transformación de constructos variádicos a una forma binaria estricta mediante el uso de funciones de orden superior. Mecánicamente, estas funciones iteran sobre la lista de argumentos desazucarados, aplicando progresivamente el constructor binario (`BinOp`) para generar una estructura anidada. Se distinguen dos estrategias críticas:

- Se emplea pliegue por la izquierda (*fold-left*) para operadores aritméticos estándar (resta, división), garantizando la asociatividad izquierda necesaria para preservar la semántica matemática $((a - b) - c)$.
- Se emplea pliegue por la derecha (*fold-right*) para casos específicos como la exponenciación (`expt`) y la construcción de listas (`cons`). Esto asegura que la evaluación procede desde el último elemento hacia el primero. $(a^{(b^c)})$, respetando la naturaleza recursiva de estas operaciones.

6. Desazucaramiento de Operadores Variádicos

6.1. El Papel Fundamental de `fold`

Una decisión de diseño crucial en LAMBDAAE es que todos los operadores son **prefijos** y se **asocian hacia la izquierda**. Esto tiene profundas implicaciones semánticas:

$$(+\ 1\ 2\ 3\ 4) \equiv (((1 + 2) + 3) + 4) \quad (\text{asociatividad izquierda})$$

Para implementar esta asociatividad, el desazucaramiento utiliza `foldl1` (*fold left*), que es esencial para:

- Preservar la semántica de asociatividad izquierda.
- Reducir operaciones n-arias a operaciones binarias.
- Mantener el orden de evaluación ansioso (*call-by-value*).

6.2. Operadores Aritméticos

```
desugar (AddListS es) = foldl1 (BinOp AddOp) (map desugar es)
desugar (SubListS es) = foldl1 (BinOp SubOp) (map desugar es)
desugar (MulListS es) = foldl1 (BinOp MulOp) (map desugar es)
desugar (DivListS es) = foldl1 (BinOp DivOp) (map desugar es)
```

Ejemplo detallado

$$(-\ 10\ 3\ 2)$$

Proceso de desazucaramiento:

1. El parser genera:

```
SubListS [NumS 10, NumS 3, NumS 2]
```

2. Aplicación de `map desugar`:

```
[Num 10, Num 3, Num 2]
```

3. Aplicación de:

```
foldl1 (BinOp SubOp)
```

Recordando que:

$$\text{foldl1 } f\ [a, b, c] = f(f\ a\ b)\ c$$

4. Resultado final:

```
BinOp SubOp (BinOp SubOp (Num 10) (Num 3)) (Num 2)
```

Representación en árbol

```

      BinOp SubOp
        /      \
    BinOp SubOp  Num 2
      /      \
    Num 10    Num 3
```

7. Importancia de `foldl1` y `foldr1`

7.1. Asociatividad Izquierda en Operadores Aritméticos

La función `foldl1` es crucial en el desazucaramiento de operadores variádicos, pues garantiza la asociatividad izquierda. Por ejemplo:

$$(+ \ 1 \ 2 \ 3 \ 4) \equiv (((1 + 2) + 3) + 4)$$

Si se utilizara `foldr1`, se obtendría una semántica incorrecta. En particular, para la resta:

$$\text{foldl1} : \quad (10 - 3) - 2 = 5$$

$$\text{foldr1} : \quad 10 - (3 - 2) = 9$$

Lo cual no coincide con la notación prefija asociativa a la izquierda establecida en LAMBDAE.

7.2. Exponenciación: Asociatividad Derecha

La exponenciación es la única operación que debe asociarse hacia la derecha. Por ello, se usa:

$$\text{foldr1} \ (\text{BinOp} \ \text{ExptOp})$$

Ejemplo:

$$(\text{expt} \ 2 \ 3 \ 2) \equiv 2^{(3^2)} = 2^9 = 512$$

Si se aplicara asociatividad izquierda:

$$(2^3)^2 = 8^2 = 64$$

lo cual sería incorrecto.

7.3. Comparaciones Encadenadas

Los operadores de comparación se traducen en comparaciones consecutivas, todas las cuales deben evaluarse (estrategia ansiosa):

$$(< \ 1 \ 2 \ 3) \equiv \text{and}(1 < 2, 2 < 3)$$

A diferencia de las operaciones aritméticas, los operadores relacionales ($<$, $>$, $=$, ...) no son cerrados sobre su dominio (reciben números pero regresan booleanos), lo que impide el uso de una reducción simple (*fold*). Por ello, la transformación se define formalmente como una cadena recursiva de conjunciones lógicas que verifican la propiedad de adyacencia:

$$\mathcal{D}(\text{op} \ e_1 \ e_2 \ \dots \ e_n) = \text{BinOp}(\text{And}, \underbrace{\text{BinOp}_{\text{op}}(\mathcal{D}e_1, \mathcal{D}e_2)}_{\text{Comparación Binaria}}, \underbrace{\mathcal{D}(\text{op} \ e_2 \ \dots \ e_n)}_{\text{Recursión}})$$

Con el caso base para un solo elemento: $\mathcal{D}(op\ e_n) = \text{Boolean}(\text{True})$. Esta expansión garantiza semánticamente que se verifiquen todos los intervalos $(e_i\ op\ e_{i+1})$ de izquierda a derecha.

8. Implementación

```
chainCompare :: Op -> [ASA] -> ASA
chainCompare _ [] = Boolean True
chainCompare _ [_] = Boolean True
chainCompare op (x:y:rest) =
  let cmp = BinOp op x y
  in case rest of
    [] -> cmp
    _ -> BinOp AndOp cmp (chainCompare op (y:rest))
```

Ejemplo:

(< 1 5 10 15)

Se desazucara a:

BinOp AndOp (BinOp LtOp 1 5) (BinOp AndOp (BinOp LtOp 5 10) (BinOp LtOp 10 15))

Este diseño evita el cortocircuito del operador **and** y garantiza evaluación completa.

La implementación concreta materializa la regla recursiva definida anteriormente, transformando la lista lineal de operandos en una jerarquía de conjunciones anidadas. Esta estructura garantiza que se verifique la relación binaria para cada par de elementos adyacentes (e_i, e_{i+1}) en la secuencia. Por ejemplo, la expresión variádica:

(< 1 5 10 15)

Se transforma sistemáticamente en el siguiente árbol de sintaxis abstracta (ASA), donde el operador lógico **And** unifica las validaciones individuales:

BinOp(And, BinOp(Lt, 1, 5), BinOp(And, BinOp(Lt, 5, 10), BinOp(Lt, 10, 15)))

Par 1-2 Par 2-3 Par 3-4

Este diseño hace explícito el flujo de evaluación, forzando la comprobación de todos los intervalos sin depender de mecanismos de cortocircuito implícitos del lenguaje anfitrión.

9. Curryng: Lambdas Multivariádicas a Unarias

9.1. Principio de Curryng

En LAMBDAE toda función del núcleo es una lambda unaria:

$\lambda(x, y, z). (+\ x\ y\ z)$

Se desazucara a:

$$\text{Fun } "x" (\text{Fun } "y" (\text{Fun } "z" (\dots)))$$

Implementación:

$$\begin{aligned} \text{desugar } (\text{FunListS } [] \text{ body}) &= \text{desugar body} \\ \text{desugar } (\text{FunListS } (x:xs) \text{ body}) &= \text{Fun } x (\text{desugar } (\text{FunListS } xs \text{ body})) \end{aligned}$$

9.2. Aplicación Multivariádica

$$(f \ 1 \ 2 \ 3)$$

Se traduce mediante asociatividad izquierda:

$$\text{App}(\text{App}(\text{App } f \ 1) \ 2) \ 3$$

La transformación de funciones y aplicaciones multivariádicas responde a la restricción del núcleo mínimo de LAMBDAE, que opera exclusivamente con aridad unaria. Formalmente, este proceso se define mediante dos reglas recursivas que garantizan la equivalencia semántica con el Cálculo Lambda:

- **Abstracción (Currying):** Se descomponen los parámetros múltiples en una cadena de funciones unarias anidadas:

$$\mathcal{D}\lambda(x_1 \dots x_n).e = \text{Fun}(x_1, \mathcal{D}\lambda(x_2 \dots x_n).e)$$

- **Aplicación (Asociatividad Izquierda):** La aplicación de argumentos se reduce secuencialmente para respetar la precedencia estándar:

$$\mathcal{D}(f \ e_1 \dots e_n) = \text{App}(\mathcal{D}(f \ e_1 \dots e_{n-1}), \mathcal{D}e_n)$$

Esta formalización asegura que cualquier construcción de alto nivel sea reducible a términos válidos del núcleo semántico sin ambigüedad.

10. Desazucaramiento de Enlaces

10.1. Let Simple

Caso unidad:

$$\text{desugar}(\text{LetS}[(x, e)] \text{ body}) = \text{App}(\text{Fun } x (\text{desugar body}))(\text{desugar } e)$$

Caso múltiple:

$$\text{let } [(x, e), (y, e')] \text{ body}$$

Se traduce a:

$$\text{App}\left(\text{App}(\text{Fun } x (\text{Fun } y \text{ body})) e\right) e'$$

La transformación de un **let** con Múltiples enlaces requiere construir simultáneamente una abstracción currificada y su correspondiente secuencia de aplicaciones. Formalmente, esto se implementa mediante la composición de dos reducciones estructurales:

1. Se aplica **foldr** sobre la lista de variables para construir el núcleo funcional anidado (**Fun**), encapsulando el cuerpo de adentro hacia afuera ($\lambda x. \lambda y. \text{body}$).
2. Se aplica **foldl** sobre la lista de expresiones para generar la cadena de aplicaciones (**App**), asegurando que la función currificada consuma sus argumentos en orden de izquierda a derecha $(\dots e) e'$. Esta estrategia garantiza que la estructura sintáctica **let** sea isomorfa a la aplicación inmediata de una lambda n-aria en el núcleo.

10.2. **let***

```
desugar (LetStarS [] body) = desugar body
desugar (LetStarS ((x,e):rest) body) =
  desugar (LetS [(x,e)] (LetStarS rest body))
```

A diferencia del **let** paralelo, la construcción **let*** permite que cada definición haga referencia a las variables vinculadas anteriormente en la misma lista. Formalmente, esto se define mediante una transformación recursiva que expande la lista de enlaces en una jerarquía de expresiones **let** anidadas (que a su vez se desazucaran a lambdas). La función de transformación $\mathcal{D}$ se define inductivamente sobre la lista de enlaces:

- **Caso Base:** Si no hay enlaces pendientes, se procesa el cuerpo directamente:

$$\mathcal{D}(\text{let}^* () \text{ body}) = \mathcal{D}\text{body}$$

- **Caso Recursivo:** Se extrae el primer par (x, e) para formar un **let** externo, cuyo cuerpo contiene el resto de los enlaces:

$$\mathcal{D}(\text{let}^* ((x, e) : \text{rest}) \text{ body}) = \mathcal{D}(\text{let } (x, e) (\text{let}^* (\text{rest}) \text{ body}))$$

Esta definición recursiva garantiza que el alcance de la variable x cubre tanto las definiciones siguientes en **rest** como el **body** final.

10.3. **letrec** y el Combinador **Z**

Traducción:

$$\text{letrec } (f \ v) \ c \quad \Rightarrow \quad \text{let } (f = Z \ (\lambda(f).v)) \ c$$

El combinador Z , apto para evaluación ansiosa, es:

$$Z = \lambda f. (\lambda x. f(\lambda v. (xx) v))(\lambda x. f(\lambda v. (xx) v))$$

Para introducir definiciones recursivas en un núcleo de cálculo lambda puro (que carece de autorreferencia nativa), la construcción **letrec** se transforma formalmente en un enlace **let** donde la función se define como el punto fijo de su abstracción. Es fundamental notar que, debido a la estrategia de evaluación ansiosa de LAMBDAE, el uso del combinador Y estándar provocaría divergencia (no terminación) inmediata. Por esta razón, la formalización implementa el Combinador Z. Este operador presenta una η -expansión amplia $(\lambda v. (xx) v)$ dentro de la autoaplicación, lo que retrasa la evaluación del paso recursivo hasta que el argumento es estrictamente necesario, garantizando así la convergencia de la definición en un entorno estricto.

11. Listas y Desazucaramiento

11.1. Construcción

$$[1, 2, 3] \equiv (\text{cons } 1 (\text{cons } 2 (\text{cons } 3 \text{ nil})))$$

Implementación:

```
desugar (ListS [])      = Id "nil"
desugar (ListS (x:xs)) =
  App (App (Id "cons") (desugar x))
    (desugar (ListS xs))
```

Con el objetivo de dotar al lenguaje de una notación intuitiva sin comprometer la parsimonia del núcleo, se decidió implementar una versión propia de listas basada en azúcar sintáctica. Dado que el núcleo (ASA) carece de un tipo de dato primitivo para secuencias, las listas se definen formalmente mediante una transformación inductiva. La notación de corchetes $[,]$ se traduce en una estructura de pares enlazados mediante las siguientes reglas recursivas:

- **Caso Base (Vacío):** La lista vacía se reduce directamente al identificador centinela:

$$\mathcal{D}[] = \text{Id}(\text{"nil"})$$

- **Caso Inductivo (Construcción):** Una lista no vacía se descompone estructuralmente, aplicando el constructor **cons** a la cabeza desazucarada y al resultado de la transformación recursiva de la cola:

$$\mathcal{D}[x : xs] = \text{App}(\text{App}(\text{Id}(\text{cons}), \mathcal{D}x), \mathcal{D}xs)$$

12. Desazucaramiento de cond

$$\text{cond } [(g_1, e_1), (g_2, e_2), \dots] \Rightarrow \text{if } g_1 \ e_1 \ (\text{if } g_2 \ e_2 \ \dots)$$

Código:

```

desugarCond [] = error "cond sin clausulas"
desugarCond [(g,e)] = desugar e
desugarCond ((g,e):rest) =
  If (desugar g) (desugar e) (desugarCond rest)

```

Ejemplo:

$$\text{cond } [(x < 0, -1), (x = 0, 0), (\text{else}, 1)]$$

$$\Rightarrow \text{if } x < 0 \text{ } -1 \text{ (if } x = 0 \text{ } 0 \text{ } 1)$$

La expresión `cond` proporciona una sintaxis para la evaluación condicional de múltiples ramas, eliminando la necesidad de anidar manualmente expresiones `if`. Formalmente, esta construcción se define como azúcar sintáctica que se reduce recursivamente a una cascada de condicionales binarios en el núcleo. La transformación $\mathcal{D}$ procesa la lista de cláusulas $\{(g_1, e_1), \dots, (g_n, e_n)\}$ de la siguiente manera:

- **Caso Recursivo :** Si existen múltiples cláusulas pendientes, se genera un nodo `If` donde la primera guardia g_1 decide si evaluar su expresión e_1 o continuar con la expansión recursiva del resto de la lista:

$$\mathcal{D}(\text{cond } (g, e) : \text{rest}) = \text{If}(\mathcal{D}g, \mathcal{D}e, \mathcal{D}(\text{cond rest}))$$

- **Caso Base (Cláusula Final/Else):** La última cláusula de la lista se interpreta como la rama "sino" (*else*) del condicional inmediatamente anterior, devolviendo su expresión directamente:

$$\mathcal{D}(\text{cond } [(g_{\text{final}}, e_{\text{final}})]) = \mathcal{D}e_{\text{final}}$$

12.1. Beneficios Arquitectónicos del Desugaring en LAMBDAE

La implementación de desugaring en LAMBDAE proporciona múltiples ventajas arquitectónicas:

12.1.1. Simplificación del Evaluador

Al reducir SASA a ASA antes de la evaluación, el intérprete de LAMBDAE solo necesita manejar un conjunto reducido de construcciones. Esto reduce significativamente la complejidad del evaluador y facilita la verificación de su correctitud.

12.1.2. Reutilización de Código y Demostraciones

Como observa el material de la Universidad de Tübingen, si se han escrito algoritmos o demostrado propiedades sobre el lenguaje núcleo (ASA), estos automáticamente se aplican a todas las construcciones del lenguaje superficial (SASA) después del desugaring. Esto representa una economía significativa en el esfuerzo de desarrollo y verificación.

12.1.3. Extensibilidad del Lenguaje

La arquitectura de dos niveles facilita la adición de nuevas características al lenguaje. Nuevas construcciones sintácticas pueden agregarse al nivel SASA siempre que puedan expresarse en términos del núcleo ASA, sin necesidad de modificar el evaluador.

12.1.4. Claridad Semántica

El desugaring hace explícita la semántica de cada construcción sintáctica al mostrar exactamente cómo se traduce en términos del núcleo. Esto proporciona una especificación formal del significado de cada característica del lenguaje.

13. Semántica operacional

La semántica operacional describe cómo se ejecutan los programas de un lenguaje paso a paso. A diferencia de la semántica denotacional (que define el significado de un programa como funciones matemáticas) o la axiomática (que define propiedades lógicas de los programas), la semántica operacional se centra en la evolución del estado del programa durante su ejecución.

Se puede ver como “un conjunto de reglas que explican cómo cada instrucción afecta al estado del programa”.

SmallStep: Semántica con ambientes

Corrección: Definición Notación de Paso Pequeño (*small-step*) MÓDULO A5

La semántica de paso pequeño (*small-step semantics*) o semántica operacional estructural, es un método formal para especificar el comportamiento de los lenguajes de programación que describe la evaluación de programas como una secuencia de pasos de reducción atómicos.

En este enfoque, se define una relación de transición (típicamente denotada como $\rightarrow$) que describe cómo una expresión o configuración del programa se reduce a otra en un único paso de cómputo. La evaluación completa de un programa se modela como una secuencia de estas transiciones individuales hasta alcanzar un valor final o un estado terminal.

Características principales:

- **Granularidad fina:** Cada paso de reducción representa la operación más pequeña de cómputo posible.
- **Determinismo o no-determinismo:** Puede especificar tanto evaluaciones deterministas como no deterministas.
- **Orden de evaluación explícito:** Hace explícito el orden en que se evalúan las subexpresiones.
- **Facilita el razonamiento:** Permite razonar sobre propiedades como la terminación, el progreso y la preservación de tipos.

Esto contrasta con la semántica de paso grande (*big-step semantics*), que describe directamente la relación entre una expresión y su resultado final sin detallar los pasos intermedios. [25–27]

A continuación se muestran las reglas de paso pequeño (*small-step semantics*²) para el lenguaje, considerando ambientes explícitos ε .

Corrección: Reglas de paso pequeño LAMBDÆ MÓDULO A5

Relación de evaluación: $\langle e, \varepsilon \rangle \longrightarrow \langle e', \varepsilon' \rangle$

Donde:

- **expresión:** es la expresión a evaluar
- ε : es el ambiente (environment) que mapea identificadores a valores
- **valor:** es el resultado de la evaluación
- ε' : es el ambiente (potencialmente modificado) después de la evaluación

13.1. Reglas Básicas

Se presentan las construcciones de las reglas básicas de nuestro lenguaje, de tal forma que cada una de estas representan los estados terminales de LAMBDAE.

Identificadores

$$\overline{\langle Id("nil"), \varepsilon \rangle} \rightarrow \langle Id("nil"), \varepsilon \rangle$$

- La lista vacía representada por el identificador especial “nil” es un valor terminal.
- En LAMBDAE, las listas vacías se representan mediante [] en la sintaxis concreta, pero en el AST (Árbol de Sintaxis Abstracta) se representan como $Id("nil")$, pues caemos en el caso en el que el identificador en el ambiente es vacío.
- Esta regla establece que `nil` ya está completamente evaluado y no requiere más reducciones.
- El ambiente ε permanece sin cambios porque no se realizan sustituciones ni búsquedas.

Números

$$\overline{\langle Num(n), \varepsilon \rangle} \rightarrow \langle Num(n), \varepsilon \rangle$$

- Un número entero n es un valor terminal según la sintaxis léxica de LAMBDAE: `int ::= 0 | [1-9][0-9]*`.
- En el AST, los números se representan mediante el constructor $Num(n)$, donde n es el valor numérico.
- Esta regla indica que los números ya están en su forma evaluada y no pueden reducirse más.
- El ambiente ε no se modifica.

Constantes booleanas

$$\overline{\langle Boolean(b), \varepsilon \rangle} \rightarrow \langle Boolean(b), \varepsilon \rangle$$

- Un valor booleano b (que puede ser `True` o `False` según la sintaxis léxica: `bool ::= true | false`) es un valor terminal.

- En el AST, los booleanos se representan mediante el constructor `Boolean(b)`, donde $b \in \{\text{True}, \text{False}\}$.
- Esta regla establece que los booleanos ya están completamente evaluados.
- El ambiente ε no se modifica.

Variables

$$\frac{\text{lookupEnv}(i, \varepsilon) = v}{\langle \text{Id}(i), \varepsilon \rangle \rightarrow \langle v, \varepsilon \rangle}$$

- Un identificador i (que según la sintaxis léxica cumple: `id ::= [A-Za-z] [A-Za-z0-9_]*`) representa una **variable** en el programa.
- En el AST, los identificadores se representan mediante el constructor `Id(i)`, donde i es el nombre de la variable.
- Definimos la construcción de la función `lookupEnv(i, ε)` busca el identificador i en el ambiente ε y regresa su valor asociado v .
- Si la búsqueda es exitosa, la expresión `Id(i)` se **reduce al valor** v que tenía almacenado en el ambiente.
- El ambiente ε no se modifica (solo se consulta).

Operaciones Binarias y Unarias

Como habíamos mencionando previamente, estas reglas implementan la estrategia de evaluación *call-by-value* (estricta) de izquierda a derecha, donde los argumentos deben evaluarse completamente antes de aplicar el operador.

Operadores Binarios (BinOp)

En LAMBDAE, los operadores binarios incluyen: `+`, `-`, `*`, `/`, `=`, `<`, `>`, `<=`, `>=`, `pair`.

Regla que comprueba si es un valor canónico:

En el caso de las operaciones binarias, contamos con la construcción *isValue*, la cual verifica si los operandos ya se encuentran en su forma canónica. Esta construcción devuelve un valor booleano: *True* indica que el operando es canónico, mientras que *False* señala que aún no lo es.

Como consecuencia, una vez que un operando es canónico, debe considerarse irreducibles, además el ambiente de evaluación no se modifica.

$$\frac{isValue(v_1) \text{ y } isValue(v_2)}{\langle BinOp(op, v_1, v_2), \varepsilon \rangle \rightarrow \langle evalBinOp(op, v_1, v_2), \varepsilon \rangle}$$

$$\frac{isValue(v_1) \quad \langle e_2, \varepsilon \rangle \rightarrow \langle e'_2, \varepsilon' \rangle}{\langle BinOp(op, v_1, e_2), \varepsilon \rangle \rightarrow \langle BinOp(op, v_1, e'_2), \varepsilon' \rangle}$$

$$\frac{\langle e_1, \varepsilon \rangle \rightarrow \langle e'_1, \varepsilon' \rangle}{\langle BinOp(op, e_1, e_2), \varepsilon \rangle \rightarrow \langle BinOp(op, e'_1, e_2), \varepsilon' \rangle}$$

Raíz Cuadrada

$$\overline{\langle Sqrt(Num(n)), \varepsilon \rangle \rightarrow \langle Num(\lfloor \sqrt{n} \rfloor), \varepsilon \rangle}$$

- El argumento es un número $Num(n)$.
- Se calcula la raíz cuadrada de n y se trunca al entero más cercano hacia abajo (función piso $\lfloor \cdot \rfloor$).
- El ambiente ε no cambia.

$$\frac{\langle e, \varepsilon \rangle \rightarrow \langle e', \varepsilon' \rangle}{\langle Sqrt(e), \varepsilon \rangle \rightarrow \langle Sqrt(e'), \varepsilon' \rangle}$$

- El argumento e no es un valor y puede dar un paso de evaluación a e' .
- Se mantiene la estructura $Sqrt$ pero se reemplaza e por e' .
- Esta regla asegura que el argumento se evalúe completamente antes de aplicar la raíz cuadrada.

Negación

$$\overline{\langle Not(Boolean(b)), \varepsilon \rangle \rightarrow \langle Boolean(\neg b), \varepsilon \rangle}$$

- El argumento es un valor booleano $Boolean(b)$, donde $b \in \{\text{True}, \text{False}\}$.
- Se aplica la negación lógica $\neg$:
 - Si $b = \text{True}$, entonces $\neg b = \text{False}$.
 - Si $b = \text{False}$, entonces $\neg b = \text{True}$.

$$\frac{\langle e, \varepsilon \rangle \rightarrow \langle e', \varepsilon' \rangle}{\langle Not(e), \varepsilon \rangle \rightarrow \langle Not(e'), \varepsilon' \rangle}$$

- El argumento e no es un valor booleano y puede dar un paso de evaluación a e' .
- Se mantiene la estructura *Not* pero se reemplaza e por e' .

Esta regla asegura que el argumento se evalúe completamente antes de aplicar la negación.

Listas y Pares

$$\frac{e = \text{Pair}(v_1, v_2)}{\langle \text{UnOp}(\text{NullOp}, e), \varepsilon \rangle \rightarrow \langle \text{Boolean}(\text{False}), \varepsilon \rangle}$$

$$\frac{\langle e, \varepsilon \rangle \rightarrow \langle e', \varepsilon' \rangle}{\langle \text{UnOp}(\text{NullOp}, e), \varepsilon \rangle \rightarrow \langle \text{UnOp}(\text{NullOp}, e'), \varepsilon' \rangle}$$

- La expresión e es un par $\text{Pair}(v_1, v_2)$, donde v_1 y v_2 son valores.
- La operación *null?* regresa *False* porque un par no es la lista vacía.
- En LAMBDAE, las listas se construyen usando pares enlazados, donde la lista vacía se representa como $\text{Id}(\text{"nil"})$.
- El ambiente ε no cambia.

Caso 2:

- El argumento e no es un valor y puede dar un paso de evaluación a e' .
- Se mantiene la estructura $\text{UnOp}(\text{NullOp}, \cdot)$ pero se reemplaza e por e' .
- Esta regla asegura que el argumento se evalúe completamente antes de verificar si es *nil*.

Pares (pair)

$$\frac{\text{isValue}(v_1) \text{ y } \text{isValue}(v_2)}{\langle \text{Pair}(v_1, v_2), \varepsilon \rangle \rightarrow \langle \text{Pair}(v_1, v_2), \varepsilon \rangle} \quad \frac{\text{isValue}(v_1) \quad \langle e_2, \varepsilon \rangle \rightarrow \langle e'_2, \varepsilon' \rangle}{\langle \text{Pair}(v_1, e_2), \varepsilon \rangle \rightarrow \langle \text{Pair}(v_1, e'_2), \varepsilon' \rangle}$$

$$\frac{\langle e_1, \varepsilon \rangle \rightarrow \langle e'_1, \varepsilon' \rangle}{\langle \text{Pair}(e_1, e_2), \varepsilon \rangle \rightarrow \langle \text{Pair}(e'_1, e_2), \varepsilon' \rangle}$$

Caso 1

- El primer componente v_1 ya es un valor.
- El segundo componente e_2 no es un valor y puede dar un paso de evaluación a e'_2 .
- Se mantiene la estructura *Pair* pero se reemplaza e_2 por e'_2 .

Esta regla implementa evaluación de izquierda a derecha: primero se evalúa v_1 , luego e_2 .

Caso 2:

- El primer componente e_1 no es un valor y puede dar un paso de evaluación a e'_1 .
- Se mantiene la estructura *Pair* pero se reemplaza e_1 por e'_1 .
- El segundo componente e_2 permanece sin cambios (aún no se evalúa).

Esta regla tiene prioridad sobre la Regla 4, asegurando evaluación de izquierda a derecha.

Primer elemento de un par(fst)

$$\frac{isValue(v_1) \text{ y } isValue(v_2)}{\langle Fst(Pair(v_1, v_2)), \varepsilon \rangle \rightarrow \langle v_1, \varepsilon \rangle} \qquad \frac{\langle e, \varepsilon \rangle \rightarrow \langle e', \varepsilon' \rangle}{\langle Fst(e), \varepsilon \rangle \rightarrow \langle Fst(e'), \varepsilon' \rangle}$$

Caso 1:

- El argumento es un par completamente evaluado $Pair(v_1, v_2)$.
- La operación *fst* extrae y retorna el primer componente v_1 del par.
- El ambiente ε no cambia.

Caso 2:

- El argumento e no es un par completamente evaluado y puede dar un paso de evaluación a e'
- Se mantiene la estructura *Fst* pero se reemplaza e por e' .
- Esta regla asegura que el argumento se evalúe completamente antes de extraer su primer componente.

Segundo elemento de un par(snd)

$$\frac{isValue(v_1) \text{ y } isValue(v_2)}{\langle Snd(Pair(v_1, v_2)), \varepsilon \rangle \rightarrow \langle v_2, \varepsilon \rangle} \qquad \frac{\langle e, \varepsilon \rangle \rightarrow \langle e', \varepsilon' \rangle}{\langle Snd(e), \varepsilon \rangle \rightarrow \langle Snd(e'), \varepsilon' \rangle}$$

Caso 1:

- El argumento es un par completamente evaluado $Pair(v_1, v_2)$.
- La operación *snd* extrae y retorna el segundo componente v_2 del par.
- El ambiente ε no cambia.

Caso 2:

- El primer componente v_1 ya es un valor.
- El segundo componente e_2 no es un valor y puede dar un paso de evaluación a e'_2 .
- Se mantiene la estructura *Pair* pero se reemplaza e_2 por e'_2 .
- Esta regla implementa evaluación de izquierda a derecha: primero se evalúa v_1 , luego e_2 .

Condicionales y Funciones

Condicional If

$$\frac{}{\langle If(Boolean(True), e_t, e_f), \varepsilon \rangle \rightarrow \langle e_t, \varepsilon \rangle}$$

- La condición ya está evaluada y es *Boolean(True)*.
- Se selecciona la rama *then* (e_t) y se descarta la rama *else* (e_f).
- La expresión se reduce directamente a e_t , que luego debe evaluarse.
- El ambiente ε no cambia.
- Esta regla implementa la semántica *lazy* para las ramas: solo se evalúa la rama seleccionada.

$$\frac{}{\langle If(Boolean(False), e_t, e_f), \varepsilon \rangle \rightarrow \langle e_f, \varepsilon \rangle}$$

- La condición ya está evaluada y es *Boolean(False)*.
- Se selecciona la rama *else* (e_f) y se descarta la rama *then* (e_t).
- La expresión se reduce directamente a e_f , que luego debe evaluarse.
- El ambiente ε no cambia.

$$\frac{\langle cond, \varepsilon \rangle \rightarrow \langle cond', \varepsilon' \rangle}{\langle If(cond, e_t, e_f), \varepsilon \rangle \rightarrow \langle If(cond', e_t, e_f), \varepsilon' \rangle}$$

- La condición *cond* no es un valor booleano y puede dar un paso de evaluación a $cond'$.
- Se mantiene la estructura *If* pero se reemplaza *cond* por $cond'$.
- Las ramas e_t y e_f permanecen sin cambios (aún no se evalúan).

- Esta regla asegura que la condición se evalúe completamente antes de seleccionar una rama.

Funciones anónimas

$$\overline{\langle Fun(p, body), \varepsilon \rangle} \rightarrow \langle Closure(p, body, \varepsilon), \varepsilon \rangle$$

- Se tiene una definición de función con parámetro p y cuerpo $body$.
- Se crea una clausura (*closure*) que captura:
 - p : el parámetro formal de la función,
 - $body$: el cuerpo de la función (expresión),
 - ε : el ambiente en el momento de la definición (captura léxica).
- Una clausura es un valor terminal que representa una función.
- El ambiente externo ε no cambia.

Esto implementa el **alcance léxico (lexical scoping)**: la función “recuerda” el ambiente donde fue definida.

Concepto clave. Una *closure* encapsula tres componentes:

- **Parámetro:** el nombre del argumento.
- **Cuerpo:** la expresión que se evaluará cuando la función sea invocada.
- **Ambiente:** las variables visibles en el momento de definición.

Aplicaciones de función(app)

$$\frac{e_1 = Closure(p, body, \varepsilon') \quad isValue(e_2)}{\langle App(e_1, e_2), \varepsilon \rangle \rightarrow \langle Expr(body, (p, e_2) : \varepsilon'), \varepsilon \rangle}$$

- e_1 es una clausura $Closure(p, body, \varepsilon')$ (función completamente evaluada).
- e_2 es un valor (argumento completamente evaluado).

Se aplica la función mediante los siguientes pasos:

1. Se toma el cuerpo de la función ($body$).
2. Se crea un nuevo ambiente extendiendo ε' (el ambiente capturado) con el enlace (p, e_2) .
3. Se evalúa $body$ en este nuevo ambiente usando la construcción $Expr$.

Nota importante: El ambiente externo ε se preserva, pero no se usa para evaluar el cuerpo.

La notación $(p, e_2) : \varepsilon'$ significa agregar el enlace “ $p \rightarrow e_2$ ” al frente del ambiente ε' .

Esta regla implementa la sustitución mediante ambientes: en lugar de reemplazar textualmente p por e_2 en *body*, se evalúa *body* en un ambiente donde p está enlazado a e_2 .

$$\frac{isValue(e_1) \quad \langle e_2, \varepsilon \rangle \rightarrow \langle e'_2, \varepsilon' \rangle}{\langle App(e_1, e_2), \varepsilon \rangle \rightarrow \langle App(e_1, e'_2), \varepsilon' \rangle}$$

- La función e_1 ya es un valor (clausura completamente evaluada).
- El argumento e_2 no es un valor y puede dar un paso de evaluación a e'_2 .

Como podemos observar se mantiene la estructura *App* pero se reemplaza e_2 por e'_2 . Esta regla implementa evaluación de izquierda a derecha: primero se evalúa la función y luego el argumento.

$$\frac{\langle e_1, \varepsilon \rangle \rightarrow \langle e'_1, \varepsilon' \rangle}{\langle App(e_1, e_2), \varepsilon \rangle \rightarrow \langle App(e'_1, e_2), \varepsilon' \rangle}$$

- La función e_1 no es un valor (no es una clausura todavía) y puede dar un paso de evaluación a e'_1 .
- Se mantiene la estructura *App* pero se reemplaza e_1 por e'_1 .
- El argumento e_2 permanece sin cambios (aún no se evalúa).

Esta regla tiene prioridad sobre la Regla 6, asegurando que primero se evalúe la función.

Evaluación de la expresión en el ambiente interno

$$\frac{isValue(e)}{\langle Expr(e, \varepsilon_{in}), \varepsilon_{out} \rangle \rightarrow \langle e, \varepsilon_{out} \rangle}$$

- La expresión e ya es un valor (completamente evaluada en el ambiente ε_{in}).
- Se regresa el valor e y se **restaura el ambiente externo** ε_{out} .

Algo importante que debemos de notar es :

- ε_{in} se usa para evaluar e (ambiente interno).
- ε_{out} se restaura después de la evaluación (ambiente externo).

Esta regla es esencial para mantener el **alcance léxico**: después de evaluar el cuerpo de una función, se vuelve al ambiente donde se hizo la llamada.

Contexto: Esta regla se usa principalmente cuando el cuerpo de una función ya se ha evaluado completamente.

$$\frac{\langle e, \varepsilon_{in} \rangle \rightarrow \langle e', \varepsilon'_{in} \rangle}{\langle Expr(e, \varepsilon_{in}), \varepsilon_{out} \rangle \rightarrow \langle Expr(e', \varepsilon'_{in}), \varepsilon_{out} \rangle}$$

- La expresión e no es un valor y puede dar un paso de evaluación a e' en el entorno ε_{in} .
- Se actualiza la expresión a e' y el ambiente interno a ε'_{in} , pero se mantiene el entorno externo ε_{out} sin cambios.
- Esta regla permite que la evaluación continúe dentro del ambiente interno hasta que se produzca un valor.

Expr actúa como un “contexto de evaluación” que permite evaluar una expresión en un ambiente específico mientras se preserva otro entorno para uso posterior.

²Semántica de paso pequeño

14. Conclusiones

Conclusiones MÓDULO D3

El desarrollo del Proyecto 1, enfocado en la formalización e implementación del lenguaje LAMBDAE (una extensión de MiniLisp), ha representado una aplicación rigurosa de los principios del diseño de lenguajes de programación. Se ha logrado definir formalmente la sintaxis léxica, la gramática libre de contexto en notación EBNF y la sintaxis abstracta (ASA). Las extensiones incorporadas, tales como operadores variádicos, estructuras de datos (pares, listas), condicionales por clausulado *cond* y funciones con múltiples parámetros, han ampliado significativamente la expresividad del lenguaje.

La implementación del proceso de *desugaring* en LAMBDAE, mediante la transformación de SASA a ASA, representa una aplicación rigurosa de principios establecidos en el diseño de lenguajes de programación. Al separar explícitamente la sintaxis superficial del núcleo semántico, LAMBDAE logra un balance entre expresividad para el programador y simplicidad para el implementador. Esta arquitectura no solo simplifica la implementación del intérprete, sino que también proporciona una base sólida para razonar formalmente sobre las propiedades del lenguaje, facilitando su extensión y mantenimiento futuro.

El proceso de quitar elementos sintácticos no esenciales, como paréntesis redundantes, y representar todas las expresiones en formas canónicas (operaciones binarias, funciones unarias, aplicación binaria) demuestra el poder del *desugaring* como herramienta de diseño de lenguajes. Como establece la literatura, esta técnica es fundamental no solo para la implementación práctica de lenguajes, sino también para su estudio teórico y su enseñanza. [12]

A pesar de la solidez de la formalización alcanzada, el lenguaje LAMBDAE no es completamente “total” y su robustez puede mejorarse, ya que ciertas complejidades (como la evaluación de comparaciones y el manejo de errores) se manejan de manera implícita o se delegan al lenguaje anfitrión (Haskell). Para avanzar hacia un lenguaje **más total y con mayor autonomía semántica**, es fundamental incluir las siguientes mejoras en el trabajo futuro:

Puntos importantes para mejorar la calidad y la totalidad de LAMBDAE

1. **Flujo Explícito para Operadores Variádicos de Comparación.** Se debe construir un mecanismo auxiliar para los operadores variádicos que se reducen a operadores binarios al comparar expresiones tales como $=$, $<$, $>$, entre otros, con el fin de asegurar un flujo explícito. Actualmente, la semántica verifica implícitamente que:

$$e_1 = e_2 \wedge e_2 = e_3 \wedge \dots \wedge e_{n-1} = e_n.$$

Para formalizarlo de forma clara, se propone **desazucarar** estas expresiones en una secuencia de conjunciones binarias. Por ejemplo, para la igualdad:

$$\text{And}(\text{Eq } e_1 e_2, \text{And}(\text{Eq } e_2 e_3, \dots)).$$

Esta transformación hace explícito el flujo de evaluación y garantiza el comportamiento de *short-circuiting*: si una comparación falla, toda la expresión resulta falsa.

2. **Manejo de Excepciones Interno para Variables Libres.** En el caso de la función `lookupEnv( $i, \varepsilon$ )`, es necesario que **el manejo de excepciones sea propio de LAMBDAE** y no delegado a Haskell. Actualmente, la terminación del programa ante una variable libre no está definida por las reglas formales del lenguaje, lo cual limita su **totalidad**. Una solución consiste en implementar una **función auxiliar** y una **regla semántica adicional** que detecte cuando una variable es libre y produzca un valor de error interno `ErrorV`. Esto permitiría que el propio lenguaje gestione sus fallos de alcance y tipado.

En conclusión, la implementación de LAMBDAE ha cumplido los objetivos de formalización y ha demostrado la viabilidad de utilizar el *desugaring* y las cerraduras para construir un intérprete funcional coherente con el alcance estático. Las extensiones futuras, centradas en la explicitación del flujo de control lógico y la **internalización de las excepciones semánticas**, constituyen el paso natural hacia la **completitud semántica** del lenguaje, permitiendo que LAMBDAE maneje todo su comportamiento —incluyendo los errores— dentro de su propio marco formal. Esto fortalece el puente entre la teoría y la práctica al diseñar un lenguaje cuyas propiedades de terminación y corrección pueden ser razonadas con mayor precisión.

El proceso de hacer un lenguaje más total, asumiendo la responsabilidad de manejar todas las condiciones posibles (incluyendo fallas de *lookup* y optimizaciones de flujo), es similar a un cocinero que, en lugar de comprar ingredientes ya procesados y depender de un proveedor externo (Haskell), decide cultivar y preparar todos sus insumos desde cero, asegurando la calidad y el control de cada componente del plato final (el programa).

15. Referencias

Referencias

- [1] LambdaSpace. (s. f.). Notas de la asignatura Lenguajes de Programación (n.º 02)[PDF]. Recuperado de https://lambdasspace.github.io/LDP/notas/ldp_n02.pdf
- [2] LambdaSpace. (s. f.). *Notas de la asignatura Lenguajes de Programación (n.º 04)* [PDF]. Recuperado de https://lambdasspace.github.io/LDP/notas/ldp_n04.pdf
- [3] LambdaSpace. (s. f.). *Notas de la asignatura Lenguajes de Programación (n.º 05)* [PDF]. Recuperado de https://lambdasspace.github.io/LDP/notas/ldp_n05.pdf
- [4] Universidad Nacional Autónoma de México, Facultad de Ciencias. (s. f.). *Material de Lenguajes de Programación*. Recuperado de <https://sites.google.com/ciencias.unam.mx/lengprog/material?authuser=0>
- [5] Universidad Nacional Autónoma de México. (2025 jul-sep). *Cálculo de Compiladores Correctos: Del Régimen Estricto al Perezoso*. Tesis / Documento DGB-UNAM. Recuperado de https://tesiunamdocumentos.dgb.unam.mx/ptd2025/jul_sep/0877653/Index.html
- [6] Aho A., Lam M., Sethi R., Ullman J., (s.f.). *Compiladores. Principios, técnicas y herramientas. Segunda Edición.*. PEARSON. Recuperado de <https://es.scribd.com/document/857303172/Compiladores>
- [7] Winskel, G. (1993). *The Formal Semantics of Programming Languages: An Introduction*. Cambridge, MA: MIT Press.
- [8] Scott, M. L. (2007). *Programming Language Pragmatics*. Amsterdam/Boston et al.: Morgan Kaufmann.
- [9] Krishnamurthi, S. (2007). *Programming Languages: Application and Interpretation*. Brown University. Recuperado de <https://cs.brown.edu/~sk/Publications/Books/ProgLangs/2007-04-26/plai-2007-04-26.pdf>
- [10] Harper, R. (s. f.). *Practical Foundations for Programming Languages*.
- [11] International Organization for Standardization & International Electrotechnical Commission. (1996). *ISO/IEC 14977:1996 – Information technology — Syntactic metalanguage — Extended BNF* (Edición 1). Ginebra: ISO.
- [12] Krishnamurthi, S. (2015). *Desugaring in Practice: Opportunities and Challenges*. PEPM 2015. Recuperado de <https://dl.acm.org/doi/10.1145/2678015.2678016>
- [13] Krishnamurthi, S. (2007). *Programming Languages: Application and Interpretation* (2.^a ed.). Brown University. Recuperado de <https://cs.brown.edu/courses/cs173/2012/book/>

- [14] Rathi, M. (2020, 1 de julio). *Desugaring – taking our high-level language and simplifying it!*. Recuperado de <https://mukulrathi.com/create-your-own-programming-language/lower-language-constructs-to-llvm/>
- [15] Serokell. (2020, 11 de agosto). *Understanding Haskell Features Through Their Desugaring*. Recuperado de <https://serokell.io/blog/haskell-to-core>
- [16] Syntactic sugar. (s. f.). En Wikipedia. Recuperado de https://en.wikipedia.org/wiki/Syntactic_sugar
- [17] IBM. (s.f.). *Fortran: The world's first programming language standard opened the door to modern computing*. IBM Heritage Center. Recuperado de <https://www.ibm.com/history/fortran>
- [18] IBM. (s.f.). *What is COBOL?* IBM Think. Recuperado de <https://www.ibm.com/think/topics/cobol>
- [19] OpenStax. (2024). *Introduction to Computer Science* (Cap. 7.1). Houston, TX: OpenStax. Recuperado de <https://openstax.org/books/introduction-computer-science/pages/7-1-programming-language-foundations>
- [20] F. E. Miranda Perea y L. D. C. González Huesca, Lenguajes de programación. notade clase 03: Sintaxis, Recuperado de <https://sites.google.com/ciencias.unam.mx/lengprog/material>, Facultad de Ciencias, UNAM., sep. de 2025.
- [21] J. C. Mitchell, Concepts in Programming Languages. Cambridge University Press, 2002
- [22] V. Alfred, Compilers: Principles, Techniques and Tools Second Edition, (2006), Columbia University, Stanford University, Recuperado de: <https://pdfcoffee.com/aho-compilers-principles-techniques-and-tools-2e-pdf-free.html>
- [23] Wikipedia, Syntax, (s.f), Recuperado de: <https://en.wikipedia.org/wiki/Syntax>
- [24] ps-tuebingen-courses. (s.f.). *Desugaring*. En *Programming Languages I*. Recuperado de: <https://ps-tuebingen-courses.github.io/pl1-lecture-notes/04-desugaring/desugaring.html>
- [25] Pierce, B. C. (2002). *Types and programming languages*. MIT Press.
- [26] Plotkin, G. D. (2004). A structural approach to operational semantics. *The Journal of Logic and Algebraic Programming*, 60-61, 17–139. <https://doi.org/10.1016/j.jlap.2004.05.001>
- [27] Wright, A. K., & Felleisen, M. (1994). A syntactic approach to type soundness. *Information and Computation*, 115(1), 38–94. <https://doi.org/10.1006/inco.1994.1093>